

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA



Báo cáo ứng dụng
Hệ thống thông minh
Used Mobile Phone Price Prediction

Giảng viên hướng dẫn: PSG.TS. Quản Thành Thơ

Học viên:

Ngô Nhất Toàn	2570471
Nguyễn Hoàng Nam	2570471
Nguyễn Huỳnh Như	2570471

THÀNH PHỐ HỒ CHÍ MINH, Tháng 4/2026

Mục lục

1	Tổng quan bài toán, bối cảnh ứng dụng và các bên liên quan	1
1.1	Bối cảnh thực tiễn của bài toán	1
1.2	Mô tả bài toán	1
1.3	Các bên liên quan	2
1.4	Giá trị và giới hạn của ứng dụng	3
2	Cơ sở lý thuyết của thuật toán và lập luận lựa chọn mô hình	3
2.1	Thuật toán cốt lõi được lựa chọn trong hệ thống	3
2.2	Random Forest Regressor: nguyên lý nền tảng	4
2.3	Tiền xử lý dữ liệu	5
2.4	Hàm mục tiêu, loss và các chỉ số đánh giá	7
2.5	Quyết định lựa chọn mô hình	8
3	Thiết kế hệ thống phần mềm và kiến trúc vận hành tổng thể	9
3.1	Tổng quan kiến trúc hệ thống	9
3.2	Các luồng nghiệp vụ chính trong hệ thống	11
3.3	Các công nghệ triển khai chính	13
4	Tập dữ liệu, phân tích đặc trưng và pipeline tiền xử lý	17
4.1	Nguồn dữ liệu và cấu trúc ngữ nghĩa ban đầu	17
4.2	Chuẩn hóa dữ liệu, xây dựng đặc trưng	18
4.3	Đặc điểm thống kê của dữ liệu, EDA và chiến lược tăng cường mẫu	19
5	Vòng đời mô hình AI và cơ chế huấn luyện	22
5.1	Tổng quan vòng đời mô hình	22
5.2	Cơ chế huấn luyện	23
5.3	Kiểm soát chất lượng mô hình và tính hợp lệ đặc trưng	24
5.4	Quản trị vòng đời mô hình và vận hành suy luận	25
6	Đánh giá kết quả	26
6.1	Khung đánh giá chất lượng mô hình	26
6.2	So sánh tổng quan các version mô hình	26
6.3	Thảo luận kết quả	28
	Phụ lục kỹ thuật	30

A	Cấu trúc hiện thực của hệ thống và quan hệ giữa các module	30
A.1	Điểm vào và lớp điều phối ứng dụng	30
A.2	Các module nghiệp vụ và hạ tầng	30
B	Rủi ro, giới hạn và kiểm soát chất lượng của hệ thống	31
B.1	Rủi ro dữ liệu và rủi ro đánh giá mô hình	31
B.2	Rủi ro vận hành, triển khai và bảo mật	32
B.3	Kiểm soát chất lượng hiện có và hướng tăng cường	33
C	Triển khai, vận hành, bảo trì và định hướng phát triển	33
C.1	Khởi chạy hệ thống và môi trường thực thi	33
C.2	Artifact vận hành và quy trình bảo trì mô hình	34
C.3	Các hướng mở rộng, nghiên cứu	34
D	Từ điển dữ liệu, ánh xạ đặc trưng và tham chiếu nghiệp vụ	35
D.1	Dữ liệu huấn luyện và ánh xạ đặc trưng	35
D.2	Dữ liệu suy luận và ràng buộc đầu vào	37
D.3	Metadata mô hình	40

1 Tổng quan bài toán, bối cảnh ứng dụng và các bên liên quan

1.1 Bối cảnh thực tiễn của bài toán

Trong thị trường điện thoại cũ, giá trị của một thiết bị không chỉ phụ thuộc vào một thông số đơn lẻ mà là kết quả tổng hợp của nhiều yếu tố như thương hiệu, cấu hình phần cứng, năm phát hành, mức hao mòn theo thời gian và kỳ vọng của thị trường đối với từng phân khúc sản phẩm. Trong thực tế, người dùng phổ thông thường định giá điện thoại bằng cảm tính hoặc theo giá tham khảo rời rạc từ các sàn giao dịch, dẫn đến hiện tượng định giá quá cao, quá thấp hoặc không nhất quán giữa các mẫu máy tương đồng.

Trong quá trình thực hiện, dự án **Mobile-Phone-Price-Prediction** được xây dựng để giải quyết chính khoảng trống này. Điểm quan trọng của hệ thống là nó không chỉ dừng ở việc tạo ra một mô hình hồi quy dự đoán giá, mà còn hiện thực toàn bộ chu trình vận hành xung quanh mô hình gồm: chuẩn bị dữ liệu, mở rộng dữ liệu bằng mẫu tổng hợp, huấn luyện mô hình, đánh giá mô hình, quản lý phiên bản, chuyển đổi mô hình active, quản lý người dùng và cung cấp giao diện web cho suy luận. Như vậy, về mặt học thuật, dự án là một ví dụ hoàn chỉnh của một hệ thống machine learning vận hành được trên dữ liệu cấu trúc; về mặt kỹ thuật, đây là một sản phẩm phần mềm có ranh giới chức năng tương đối rõ ràng, phục vụ cả hai lớp nhu cầu: vận hành ứng dụng và vận hành mô hình.

1.2 Mô tả bài toán

Bài toán cốt lõi của hệ thống là **bài toán hồi quy có giám sát** với mục tiêu ước lượng giá bán lại của điện thoại di động. Biến đầu vào được hình thành từ các đặc trưng cấu trúc thiết bị.

Từ góc độ nghiệp vụ, hệ thống được thiết kế để giải một bài toán hỗ trợ ra quyết định định giá trong bối cảnh giao dịch điện thoại cũ. Cụ thể, hệ thống hướng tới ba mục tiêu nghiệp vụ lớn:

1. Cung cấp một mức giá tham chiếu cho người dùng cuối khi họ có nhu cầu mua hoặc bán điện thoại cũ.
2. Cung cấp cho người quản trị và nhà phân tích dữ liệu một công cụ thực nghiệm để bổ sung dữ liệu, huấn luyện lại mô hình và so sánh các phiên bản.
3. Thiết lập một quy trình tối thiểu của MLOps ở quy mô nhỏ: dữ liệu $\rightarrow$ mô hình $\rightarrow$ đánh giá $\rightarrow$ triển khai $\rightarrow$ thay mô hình active.

Bài toán này có tính đại diện tốt cho các ứng dụng định giá tài sản số hoặc hàng hóa đã qua sử dụng, nơi đầu ra phụ thuộc mạnh vào đặc trưng cấu hình và thời gian sử dụng, nhưng dữ liệu thị trường thường nhiễu và không cân bằng.

1.3 Các bên liên quan

Người bán điện thoại đã qua sử dụng

Nhóm người bán là bên liên quan hưởng lợi trực tiếp nhất. Trong thực tế, người bán thường có ba khó khăn: không biết giá hợp lý, không biết mức khấu hao hợp lý theo thời gian, và không biết cấu hình hiện có của máy đang nằm ở phân khúc nào so với mặt bằng thị trường. Một mô hình định giá tốt có thể giảm đáng kể độ lệch trong định giá ban đầu. Nếu giá niêm yết ban đầu hợp lý hơn, xác suất giao dịch thành công tăng lên, thời gian thương lượng giảm xuống và trải nghiệm giao dịch cũng tốt hơn.

Lợi ích định lượng tiềm năng có thể được hiểu như sau: giả sử sai lệch định giá thủ công trung bình của người dùng không có công cụ hỗ trợ là ϵ_{manual} và sai lệch trung bình khi tham khảo mô hình là ϵ_{model} , khi đó lợi ích kỹ thuật cơ bản là

$$\Delta\epsilon = \epsilon_{manual} - \epsilon_{model}.$$

Nếu $\Delta\epsilon > 0$, hệ thống thực sự đang cải thiện chất lượng định giá.

Người mua thiết bị cũ:

Người mua là nhóm hưởng lợi ở phía ngược lại của giao dịch. Trong thị trường hàng cũ, người mua thường bị bất lợi thông tin, nhất là khi người bán không minh bạch về cấu hình, mức hao mòn hoặc mức chênh giữa giá đăng bán và giá trị thực. Một hệ thống dự đoán giá có thể giúp người mua hình thành khoảng giá chấp nhận được trước khi ra quyết định, từ đó giảm rủi ro mua quá giá trị thực. Nếu được triển khai rộng hơn, hệ thống còn có thể hỗ trợ chức năng sàng lọc nhanh nhiều lựa chọn trong cùng một mức ngân sách.

Data scientist hoặc kỹ sư ML vận hành hệ thống:

Đây là nhóm sử dụng các chức năng sâu hơn của dashboard. Trong hệ thống, data scientist được cấp quyền xem thống kê dữ liệu, sinh dữ liệu tổng hợp, huấn luyện mô hình mới với các siêu tham số khác nhau, chọn tập đặc trưng tùy biến, nhập thêm bản ghi bằng tay hoặc bằng CSV, xem các version mô hình và đổi model active. Như vậy, hệ thống đóng vai trò như một sandbox vận hành mô hình học máy ở quy mô nhỏ, nơi các thí nghiệm có thể được chuyển hóa thành một phiên bản mô hình có thể sử dụng ngay cho suy luận.

Quản trị viên hệ thống:

Quản trị viên không trực tiếp can thiệp vào thuật toán, nhưng họ đảm bảo tính ổn định của hệ thống thông qua các chức năng người dùng: thêm tài khoản, sửa vai trò, phân quyền, xóa tài khoản, kiểm soát số lượng admin còn lại. Đây là một vai trò quan trọng trong các hệ thống triển khai thực tế, vì mô hình học máy chỉ là một thành phần; để mô hình phục vụ được người dùng, cần có quản trị truy cập và chính sách sử dụng hợp lệ.

Cửa hàng thu mua điện thoại cũ (bên liên quan gián tiếp):

Sử dụng ứng dụng như một lớp tham chiếu ban đầu trong bước báo giá.

1.4 Giá trị và giới hạn của ứng dụng

Giá trị cốt lõi mà hệ thống mang lại

Giá trị đầu tiên là **chuẩn hóa định giá**. Thay vì định giá bằng cảm nhận chủ quan, người dùng có thêm một hàm ánh xạ từ cấu hình thiết bị sang khoảng giá dự kiến. Giá trị thứ hai là **tính tái huấn luyện**. Hệ thống không khóa mô hình ở một lần huấn luyện duy nhất; người dùng có thể bổ sung dữ liệu mới rồi huấn luyện lại để phản ánh phân phối mới của thị trường. Giá trị thứ ba là **khả năng kiểm soát vòng đời mô hình**. Việc lưu version, metadata, feature importance và active model giúp quá trình vận hành minh bạch hơn. Giá trị thứ tư là **chi phí triển khai thấp**. Vì hệ thống chạy localhost bằng Flask, không cần dịch vụ nền phức tạp, rất phù hợp cho mục tiêu học tập, nghiên cứu, demo kỹ thuật hoặc vận hành nội bộ.

Mức độ phù hợp của bài toán với machine learning

Không phải mọi bài toán định giá đều thích hợp với học máy. Một bài toán phù hợp cần có các điều kiện: tồn tại dữ liệu lịch sử, đầu ra có thể xem như hàm của một tập đặc trưng đầu vào, và có đủ động lực để mô hình hóa các tương tác phức tạp giữa các biến. Dự án này thỏa cả ba điều kiện trên. Dữ liệu có sẵn ở Kaggle; các đặc trưng phần cứng và thời gian sử dụng rõ ràng có liên hệ tới giá; và mối quan hệ giữa thương hiệu, cấu hình, đời máy và khấu hao là phi tuyến, vượt ra ngoài khả năng của các công thức tuyến tính đơn giản.

Phạm vi và giới hạn của hệ thống

Mặc dù hữu ích, hệ thống vẫn chỉ là một mô hình hỗ trợ quyết định, không phải một công cụ định giá tuyệt đối. Ở phiên bản hiện tại, hệ thống chưa xét tới các biến nghiệp vụ quan trọng như tình trạng ngoại hình, số lần thay pin, độ hiếm của máy, khu vực giao dịch, nhu cầu thị trường ngắn hạn hoặc sự kiện ra mắt model mới. Do đó, giá dự đoán phải được hiểu là *giá tham chiếu dựa trên cấu hình kỹ thuật và tuổi đời*, không phải là giá cuối cùng của mọi giao dịch.

2 Cơ sở lý thuyết của thuật toán và lập luận lựa chọn mô hình

2.1 Thuật toán cốt lõi được lựa chọn trong hệ thống

Trong hệ thống hiện thực, thuật toán học máy cốt lõi được lựa chọn là **Random Forest Regressor**. Hệ thống sử dụng lớp RandomForestRegressor của scikit-learn và đặt nó ở bước cuối của một Pipeline. Điều này có nghĩa kiến trúc mô hình thực tế

gồm hai lớp khái niệm: *tiền xử lý đặc trưng* và *mô hình hồi quy tổ hợp*. Nói cách khác, khi thảo luận về “mô hình” của dự án, cần hiểu mô hình theo nghĩa rộng là

$$\mathcal{M}(\mathbf{x}) = g(\phi(\mathbf{x})),$$

trong đó ϕ là phép biến đổi đặc trưng, còn g là rừng ngẫu nhiên dùng để hồi quy.

Phát biểu hình thức của bài toán hồi quy

Cho tập dữ liệu huấn luyện

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N,$$

trong đó $\mathbf{x}_i \in \mathbb{R}^d$ là vector đặc trưng của thiết bị thứ i , còn $y_i \in \mathbb{R}_+$ là giá bán lại thực tế. Mục tiêu là học một hàm dự đoán

$$f : \mathbb{R}^d \rightarrow \mathbb{R}_+$$

nhằm ước lượng

$$\hat{y}_i = f(\mathbf{x}_i)$$

để sai số giữa $\hat{y}_i$ và y_i nhỏ nhất theo một họ tiêu chuẩn đánh giá phù hợp.

Trong dữ liệu huấn luyện của hệ thống, cột mục tiêu được lưu ở dạng

$$\text{normalized_used_price} = \log(y),$$

nhưng khi chuẩn bị biến mục tiêu, hệ thống thực hiện

$$y = \exp(\text{normalized_used_price}).$$

Cách tổ chức này tạo ra hai lớp biểu diễn nhất quán: dữ liệu được lưu theo miền log để ổn định phân phối và hỗ trợ EDA, còn mô hình dự đoán trong miền giá thực để các chỉ số MAE, RMSE giữ nguyên ý nghĩa nghiệp vụ.

2.2 Random Forest Regressor: nguyên lý nền tảng

Random Forest là phương pháp tổ hợp dựa trên nhiều cây quyết định độc lập tương đối. Mỗi cây được huấn luyện trên một mẫu bootstrap của dữ liệu, và tại mỗi bước tách nút chỉ xét một tập con đặc trưng ngẫu nhiên. Trong bài toán hồi quy, đầu ra cuối cùng được tính bằng trung bình đầu ra của tất cả cây:

$$\hat{y}(\mathbf{x}) = \frac{1}{T} \sum_{t=1}^T h_t(\mathbf{x}),$$

trong đó T là số lượng cây, và h_t là hàm dự đoán của cây thứ t .

Với một cây hồi quy đơn lẻ, tại mỗi nút, thuật toán cố gắng tìm phép chia $S \rightarrow (S_L, S_R)$ sao cho tổng độ không thuần khiết sau chia là nhỏ nhất. Độ không thuần khiết trong hồi quy thường được đo bằng tổng bình phương sai số trong nút:

$$I(S) = \sum_{(\mathbf{x}_i, y_i) \in S} (y_i - \bar{y}_S)^2,$$

trong đó $\bar{y}_S$ là trung bình của các giá trị mục tiêu trong nút. Một phép chia tốt tối đa hóa phần giảm độ không thuần khiết:

$$\Delta I = I(S) - I(S_L) - I(S_R).$$

Vì nhiều cây được huấn luyện trên các mẫu dữ liệu hơi khác nhau và tập đặc trưng hơi khác nhau, trung bình của chúng làm giảm phương sai tổng thể. Đây là cơ chế chính khiến Random Forest thường tổng quát hóa tốt hơn một cây đơn.

Giải thích tại sao mô hình cây phù hợp với dữ liệu bảng

Dữ liệu của dự án là dữ liệu bảng hỗn hợp, gồm biến số liên tục như RAM, dung lượng pin, kích thước màn hình; biến đếm hay bán rời rạc như năm phát hành, số ngày đã dùng; và biến phân loại như hãng hoặc hệ điều hành. Đối với loại dữ liệu này, Random Forest có nhiều lợi thế:

- Không yêu cầu quan hệ tuyến tính giữa đặc trưng và đầu ra.
- Tự động học các ngưỡng quyết định như “RAM lớn hơn 6GB” hay “số ngày sử dụng lớn hơn một mức nào đó”.
- Tương đối bền vững khi dữ liệu có nhiễu hoặc một số biến thiếu giá trị.
- Hỗ trợ suy luận về độ quan trọng của đặc trưng.
- Không đòi hỏi chuẩn hóa dữ liệu mạnh như các mô hình tối ưu theo gradient.

Nếu so sánh với hồi quy tuyến tính, Random Forest biểu diễn tốt hơn các hiệu ứng phi tuyến và tương tác chéo. Nếu so sánh với mạng sâu, Random Forest phù hợp hơn trong bối cảnh dữ liệu không quá lớn, số chiều đầu vào vừa phải và mục tiêu là tính ổn định, dễ vận hành, dễ giải thích.

2.3 Tiền xử lý dữ liệu

Mô hình không trực tiếp nhận toàn bộ dữ liệu thô. Thay vào đó, hệ thống xây dựng một `ColumnTransformer` chia đặc trưng thành hai nhóm: đặc trưng phân loại và đặc trưng số.

Với nhóm phân loại, pipeline là:

$$\phi_{cat} = \text{OneHotEncoder} \circ \text{Imputer}_{mode}.$$

Bước điền giá trị mode giúp loại bỏ thiếu dữ liệu mà vẫn giữ một nhãn hợp lệ, còn one-hot encoding ánh xạ một biến phân loại nhiều giá trị thành vector nhị phân. Nếu biến **brand** có K nhãn khác nhau, phép mã hóa one-hot tạo ra vector

$$\mathbf{e}_k \in \{0, 1\}^K$$

với đúng một phần tử bằng 1.

Với nhóm số, pipeline là:

$$\phi_{num} = \text{Imputer}_{median}.$$

Việc dùng trung vị thay vì trung bình giúp giảm ảnh hưởng của các outlier và phù hợp với dữ liệu thị trường có đuôi phân phối dài.

Toàn bộ phép biến đổi đầu vào là

$$\phi(\mathbf{x}) = [\phi_{cat}(\mathbf{x}_{cat}), \phi_{num}(\mathbf{x}_{num})].$$

Sau đó, rừng ngẫu nhiên làm việc trên không gian đặc trưng đã được chuyển đổi này.

Vai trò của từng đặc trưng trong bài toán

Có thể diễn giải ý nghĩa học thuật của từng đặc trưng mặc định như sau:

- **brand**: đại diện cho định vị thị trường, giá trị thương hiệu và kỳ vọng người dùng.
- **ram**: phản ánh năng lực đa nhiệm và phân khúc cấu hình.
- **storage**: phản ánh dung lượng lưu trữ, ảnh hưởng trực tiếp đến mức giá niêm yết và giá bán lại.
- **battery_capacity**: phản ánh mức độ hữu dụng về thời lượng pin.
- **screen_size**: đại diện cho phân khúc thiết kế và trải nghiệm sử dụng.
- **camera_mp**: đại diện gần đúng cho khả năng chụp ảnh, một yếu tố marketing và trải nghiệm quan trọng.
- **release_year**: phản ánh thế hệ công nghệ.
- **days_used**: phản ánh khấu hao theo thời gian sử dụng thực.

Trong ngôn ngữ kinh tế kỹ thuật, các biến này cùng nhau mô hình hóa *giá trị chức năng*, *giá trị thương hiệu* và *khấu hao thời gian* của thiết bị.

Target leakage và quyết định loại biến khỏi tập mặc định

Trong quá trình thiết kế tập đặc trưng mặc định, `normalized_new_price` được loại ra dù dữ liệu vẫn có cột này. Lý do là giá máy mới thường có tương quan rất cao với giá máy cũ. Nếu dùng trực tiếp biến này, mô hình có thể đạt kết quả đánh giá tốt nhưng phụ thuộc vào thông tin mà trong nhiều kịch bản triển khai thực tế khó thu được hoặc quá gần với mục tiêu cần dự đoán.

Về mặt thống kê, nếu tồn tại một đặc trưng z sao cho

$$\text{corr}(z, y) \approx 1,$$

nhưng z không đại diện cho thông tin đầu vào hợp lệ tại thời điểm suy luận, thì việc giữ z sẽ làm đánh giá mô hình bị lạc quan hóa. Việc loại biến này khỏi tập mặc định nhằm giữ tính hợp lệ của pipeline học máy ở thời điểm suy luận.

2.4 Hàm mục tiêu, loss và các chỉ số đánh giá

Mặc dù hệ thống không cài đặt loss tùy biến, Random Forest Regressor vẫn tối ưu chất lượng tách nhánh theo tiêu chuẩn sai số bình phương. Sau huấn luyện, hệ thống đánh giá mô hình bằng ba chỉ số chính là MAE, RMSE và R^2 . Bộ ba này được chọn để kết hợp đồng thời thang đo sai số tuyệt đối, thang đo nhạy với lỗi lớn và một chỉ số phản ánh mức độ giải thích phương sai tổng thể.

Sai số tuyệt đối trung bình được định nghĩa bởi:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|.$$

Trong công thức trên, n là số quan sát trong tập đánh giá; y_i là giá trị thực của mẫu thứ i ; $\hat{y}_i$ là giá trị dự đoán tương ứng của mô hình; và $|y_i - \hat{y}_i|$ là độ lớn sai số tuyệt đối tại mẫu đó. Chỉ số này giữ cùng đơn vị với biến mục tiêu, tức đơn vị giá tiền. Vì vậy, MAE là chỉ số trực tiếp nhất để diễn giải mức sai lệch trung bình của mô hình trong bài toán định giá. Tuy nhiên, MAE không nhấn mạnh đủ các lỗi cực lớn, nên cần được đọc cùng RMSE.

Căn sai số bình phương trung bình được định nghĩa bởi:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}.$$

Trong đó, n , y_i và $\hat{y}_i$ giữ nguyên ý nghĩa như ở công thức MAE; $(y_i - \hat{y}_i)^2$ là sai số bình phương tại mẫu thứ i ; phép căn bậc hai ở bước cuối giúp đưa chỉ số trở về cùng đơn vị với biến mục tiêu để thuận tiện cho diễn giải. Do sai số được bình phương trước

khi trung bình, RMSE nhạy cảm hơn nhiều với những trường hợp lỗi lớn. Vì vậy, RMSE phản ánh tốt hơn rủi ro xuất hiện các ngoại lệ dự đoán nghiêm trọng.

Hệ số xác định được định nghĩa bởi:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}.$$

Ở đây, n là số quan sát của tập đánh giá; y_i là giá trị thực của mẫu thứ i ; $\hat{y}_i$ là giá trị dự đoán của mô hình; và $\bar{y}$ là giá trị trung bình của biến mục tiêu trên tập đánh giá. Tử số $\sum_{i=1}^n (y_i - \hat{y}_i)^2$ biểu diễn tổng bình phương sai số còn lại của mô hình, trong khi mẫu số $\sum_{i=1}^n (y_i - \bar{y})^2$ biểu diễn tổng phương sai ban đầu của dữ liệu so với giá trị trung bình. Nếu R^2 gần 1, mô hình giải thích được phần lớn phương sai của dữ liệu mục tiêu; nếu R^2 gần 0, mô hình không tốt hơn nhiều so với việc luôn dự đoán trung bình.

Ba chỉ số này bổ sung cho nhau: MAE cho biết lỗi trung bình dễ diễn giải, RMSE nhấn mạnh sai số lớn, còn R^2 đo mức độ giải thích phương sai mục tiêu của mô hình. Vì vậy, việc đánh giá không nên dựa trên một chỉ số đơn lẻ mà cần đọc đồng thời cả ba thước đo.

Siêu tham số và vai trò điều chuẩn

- **n_estimators**

Khi tăng **n_estimators**, phương sai của tổ hợp thường giảm và mô hình ổn định hơn, nhưng chi phí huấn luyện và suy luận tăng.

- **max_depth**

Khi giới hạn **max_depth**, mỗi cây ít phức tạp hơn, làm giảm nguy cơ overfitting

- **min_samples_split** và **min_samples_leaf**

Hai tham số này giúp kiểm soát kích thước tối thiểu của các nhánh và lá, giúp tránh các phân hoạch quá nhỏ vốn ghi nhớ nhiều của dữ liệu.

2.5 Quyết định lựa chọn mô hình

Trong quá trình lựa chọn mô hình, ba trục được dùng để cân nhắc là *độ phù hợp dữ liệu*, *khả năng triển khai* và *khả năng giải thích*. Trên ba trục đó, Random Forest được chọn vì vẫn nắm bắt được quan hệ phi tuyến, vẫn huấn luyện ổn định trên CPU cục bộ và vẫn cho phép trích xuất feature importance để phục vụ diễn giải kỹ thuật.

Nếu dùng mô hình tuyến tính, tính diễn giải có thể cao hơn nhưng sức biểu đạt sẽ thấp hơn. Nếu dùng boosting phức tạp hơn như XGBoost hoặc LightGBM, độ chính xác có thể tăng nhưng kiến trúc triển khai, dependency và cơ chế quản trị mô hình cũng sẽ phức tạp hơn. Nếu dùng deep learning, chi phí huấn luyện, tuning và giải thích sẽ tăng

manh trong khi chưa chắc tương xứng với quy mô dữ liệu hiện có. Vì vậy, trong bối cảnh triển khai hiện tại, Random Forest được giữ làm mô hình lõi của pipeline.

Những giới hạn nội tại của thuật toán

Dù phù hợp, Random Forest vẫn có hạn chế. Mô hình suy luận bằng trung bình của nhiều cây nên khó diễn giải ở mức vi mô hơn các mô hình tuyến tính. Ngoài ra, nếu phân phối dữ liệu thay đổi mạnh theo thời gian hoặc xuất hiện nhiều model mới ngoài vùng dữ liệu đã học, mô hình sẽ có xu hướng nội suy hơn là ngoại suy chính xác. Vì thế, chất lượng của mô hình phụ thuộc đáng kể vào việc cập nhật dữ liệu và tái huấn luyện định kỳ.

3 Thiết kế hệ thống phần mềm và kiến trúc vận hành tổng thể

3.1 Tổng quan kiến trúc hệ thống

Hệ thống được xây dựng như một ứng dụng web nguyên khối nhẹ trên nền Flask. Lựa chọn này nhằm giảm số lượng thành phần phải triển khai, giữ chi phí vận hành ở mức thấp và tập trung nỗ lực phát triển vào mối liên hệ giữa giao diện, logic nghiệp vụ và mô hình học máy. Về bản chất, kiến trúc hiện tại mang đặc trưng của một *modular monolith*: toàn bộ thành phần cùng chạy trong một tiến trình ứng dụng, nhưng vẫn được phân tách thành các mô-đun theo chức năng.

Nếu nhìn theo quan điểm MVC mở rộng, kiến trúc có thể được mô tả qua bốn lớp chính. Lớp *view* gồm các template trong `app/templates`. Lớp *controller* gồm các route Flask và các mô-đun điều phối trong `app/controllers`. Lớp *model* tập trung ở `PhonePriceModel` cùng các tiện ích dữ liệu liên quan. Cuối cùng là lớp cấu hình và dữ liệu, bao gồm settings, credentials, catalog thiết bị, CSV dataset, JSON state và các model pickle.

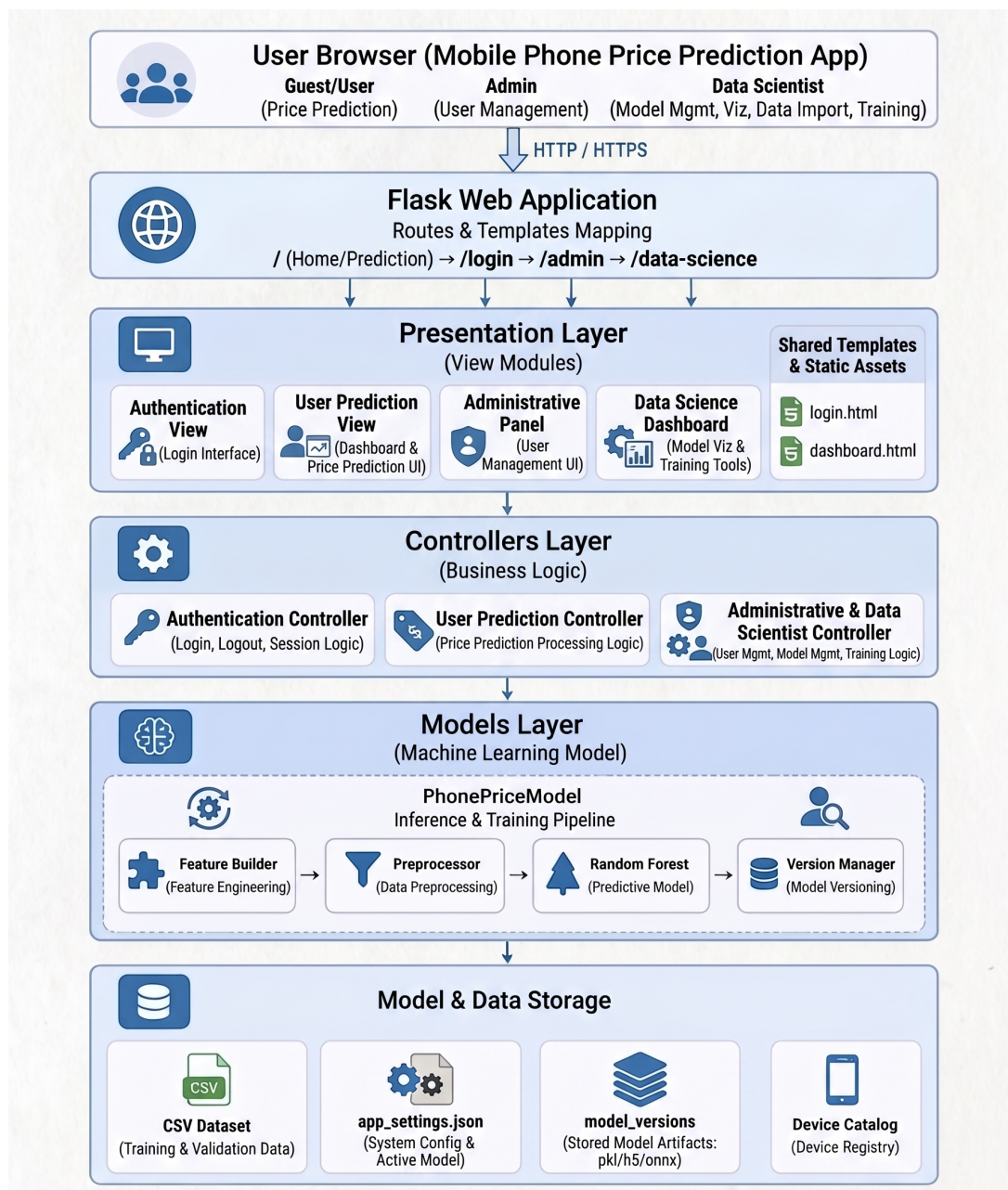


Figure 3.1: Sơ đồ kiến trúc tổng thể của hệ thống

- **View Layer:** Lớp giao diện gồm các template hiển thị cho người dùng, tiêu biểu như `app/templates/login.html` và `app/templates/dashboard.html`. Trong đó, `dashboard.html` được sử dụng như một template dùng chung cho cả Admin và Data Scientist. Nội dung hiển thị được điều chỉnh động theo role và theo section đang được truy cập, nhờ đó lớp giao diện vẫn bảo đảm được sự tách bạch về mặt trình diễn mà không cần tách thành quá nhiều file template riêng.
- **Controller Layer:** Lớp điều phối bao gồm hai nhóm thành phần chính: cơ chế định tuyến và kiểm soát quyền truy cập trong `app/app.py`, cùng với các mô-đun nghiệp vụ

trong thư mục `app/controllers`. Cụ thể, `app/app.py` đảm nhiệm việc định tuyến request, kiểm tra session role và điều phối người dùng đến đúng chức năng tương ứng. Trong nhóm controller chuyên biệt, `app/controllers/auth_controller.py` xử lý xác thực và thông tin người dùng, `app/controllers/admin_controller.py` phụ trách logic quản lý model, dataset, visualization và các thao tác CRUD người dùng, trong khi `app/controllers/user_controller.py` đảm nhiệm logic dự đoán giá.

- **Model Layer:** Lớp mô hình tập trung vào thành phần học máy và các tiện ích dữ liệu đi kèm, với hạt nhân là `app/models/phone_price_model.py` và lớp `PhonePriceModel`. Đây là nơi thực hiện các chức năng chính như xây dựng dataset, huấn luyện mô hình Random Forest, dự đoán giá và quản lý phiên bản mô hình. Lớp này đồng thời chứa các logic hỗ trợ tiền xử lý dữ liệu và đánh giá mô hình, đảm bảo rằng pipeline huấn luyện và pipeline suy luận có thể dùng chung một lõi xử lý.
- **Configuration & Data Layer:** Lớp cấu hình và dữ liệu chứa các nguồn dữ liệu, cấu hình hệ thống và các artifact phục vụ vận hành. Các thành phần tiêu biểu gồm `data/sample_phone_data.csv` cho dữ liệu huấn luyện, `app/app_settings.json` cho cấu hình ứng dụng và trạng thái active model, `app/config/credentials.py` cho thông tin credentials và phân quyền mặc định, `app/config/device_catalog.py` cho bản đồ model và brand thiết bị, cùng với thư mục `app/model_versions/` dùng để lưu metadata phiên bản mô hình và các artifact pickle.

Xét theo luồng vận hành, yêu cầu từ người dùng được tiếp nhận tại lớp giao diện, sau đó chuyển vào lớp điều phối để kiểm tra quyền truy cập và điều hướng chức năng. Từ đây, dữ liệu được đưa đến lớp mô hình để thực hiện suy luận giá hoặc huấn luyện lại mô hình, đồng thời sử dụng lớp cấu hình và dữ liệu để nạp catalog thiết bị, đọc dữ liệu huấn luyện, truy xuất metadata phiên bản và xác định mô hình đang hoạt động.

3.2 Các luồng nghiệp vụ chính trong hệ thống

Luồng nghiệp vụ thứ nhất là suy luận trực tiếp từ giao diện. Khi người dùng gửi form tại route gốc, dữ liệu từ `request.form` được ánh xạ qua hàm xây dựng đặc trưng runtime. Tại đây, model nhập vào được đối chiếu với catalog thiết bị, dung lượng pin hiệu dụng được suy ra từ battery health, và toàn bộ đầu vào được dựng lại thành một vector đặc trưng phù hợp với version mô hình đang hoạt động. Sau đó, dự đoán được trả về giao diện như một mức giá tham chiếu.

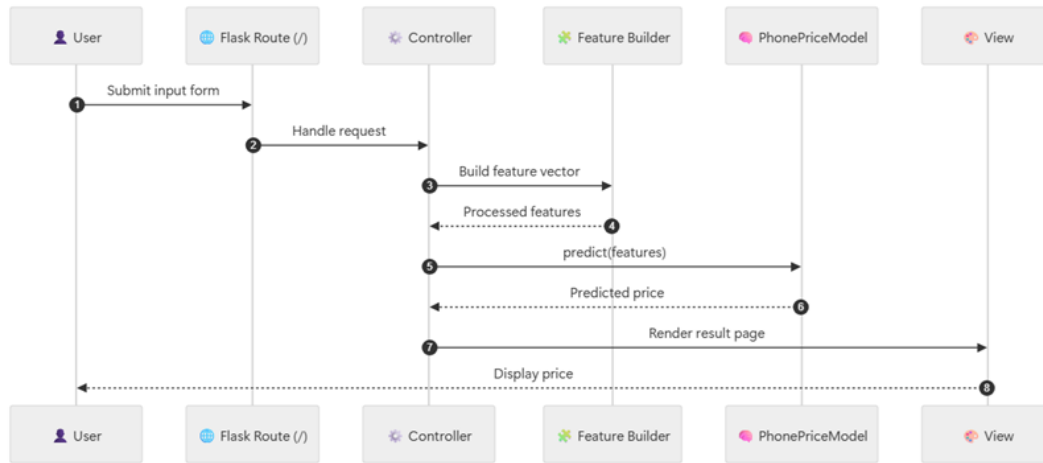


Figure 3.2: Luồng nghiệp vụ dự đoán giá điện thoại

Luồng nghiệp vụ thứ hai là quản lý mô hình dành cho data scientist. Từ dashboard `/data-science`, có thể xem thống kê dữ liệu, nhập thêm bản ghi thủ công hoặc CSV, sinh dữ liệu tổng hợp, khởi tạo huấn luyện với siêu tham số mới, so sánh các version, đổi active model hoặc xóa một version không còn dùng. Toàn bộ phần này được nối về lớp mô hình trung tâm để giữ cho pipeline huấn luyện và pipeline suy luận vẫn dùng chung logic dữ liệu.

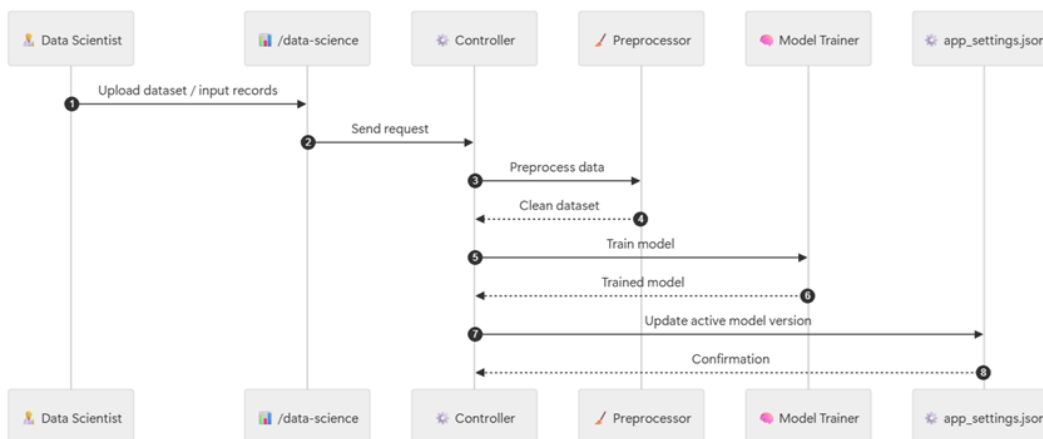


Figure 3.3: Luồng nghiệp vụ quản lý và huấn luyện mô hình

Luồng nghiệp vụ thứ ba là quản lý người dùng dành cho admin. Tại `/admin`, phần quản trị tài khoản được giữ tách biệt với dữ liệu và mô hình. Admin có thể thêm, sửa, xóa người dùng, nhưng không trực tiếp thao tác với artifact học máy. Cách tách vai trò này giữ cho phần quản trị tài khoản không lẫn với phần vận hành machine learning.

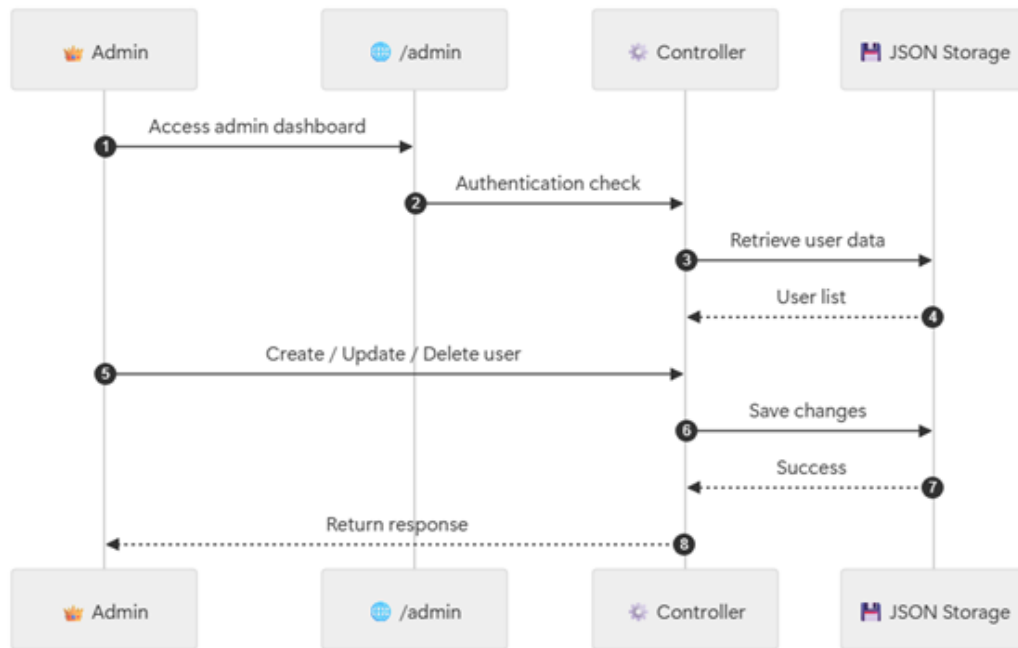


Figure 3.4: Luồng nghiệp vụ quản trị người dùng của admin

3.3 Các công nghệ triển khai chính

Stack công nghệ của hệ thống được lựa chọn theo định hướng nhất quán và phù hợp với phạm vi triển khai của đề tài. Mỗi công nghệ trong stack đảm nhiệm một vai trò riêng trong quá trình xây dựng, huấn luyện, vận hành và đánh giá hệ thống.

* Flask



Flask là web framework trung tâm của hệ thống, phù hợp với các ứng dụng cần cấu trúc gọn, linh hoạt và dễ kiểm soát luồng xử lý. Với đặc điểm nhẹ nhưng mở rộng tốt, Flask đóng vai trò kết nối giữa giao diện, logic nghiệp vụ và mô hình học máy trong cùng một ứng dụng.

Ứng dụng trong hệ thống:

- Tạo web app trung tâm cho toàn bộ hệ thống dự đoán giá điện thoại.
- Định nghĩa các route chính như /, /login, /admin và /data-science.

- Quản lý session đăng nhập, điều hướng theo vai trò người dùng và cơ chế **flash** để phản hồi thao tác.
- Render giao diện HTML server-side cho guest, admin và data scientist.

* Python



Python là ngôn ngữ triển khai thống nhất của toàn bộ hệ thống. Điểm mạnh của Python nằm ở cú pháp dễ đọc, hệ sinh thái thư viện rộng và khả năng kết nối mượt giữa lập trình web, xử lý dữ liệu và học máy.

Ứng dụng trong hệ thống:

- Triển khai toàn bộ backend, controller logic và dữ liệu runtime của ứng dụng.
- Bao phủ các bước xử lý dữ liệu, huấn luyện mô hình, versioning và suy luận.
- Cho phép kết nối liền mạch giữa Flask, pandas, scikit-learn, joblib và các thư viện trực quan hóa.

* scikit-learn



scikit-learn là thư viện học máy trung tâm trong hệ thống, cung cấp đầy đủ các thành phần để xây dựng pipeline huấn luyện và suy luận. Trong đề tài này, thư viện được dùng như lõi triển khai cho mô hình hồi quy Random Forest và cho các bước tiền xử lý đặc trưng.

Ứng dụng trong hệ thống:

- Cung cấp **RandomForestRegressor** cho bài toán dự đoán giá.
- Cung cấp **ColumnTransformer**, **SimpleImputer**, **OneHotEncoder** và **Pipeline** để xây dựng pipeline tiền xử lý và huấn luyện nhất quán.

- Hỗ trợ `train_test_split` và các metric đánh giá như MAE, RMSE và R^2 .
- Cho phép tái sử dụng cùng một logic dữ liệu giữa pha training và pha inference.

* pandas và NumPy



`pandas` là thư viện xử lý dữ liệu bảng, còn `NumPy` cung cấp nền tảng tính toán số hiệu năng cao. Hai thư viện này thường đi cùng nhau trong các hệ thống machine learning, phụ trách tổ chức dữ liệu dạng bảng và hỗ trợ các phép toán số, biến đổi mảng dữ liệu.

Ứng dụng trong hệ thống:

- Đọc, hợp nhất, chuẩn hóa và ghi dữ liệu huấn luyện từ các tệp CSV.
- Hỗ trợ nhập bản ghi thủ công và import dữ liệu hàng loạt từ dashboard quản trị dữ liệu.
- Thực hiện các phép biến đổi đặc trưng, thao tác số và chuẩn bị dữ liệu đầu vào cho pipeline học máy.
- Hỗ trợ xử lý dữ liệu batch trong các tình huống mở rộng hoặc tiền xử lý phân tích.

* SciPy



`SciPy` là thư viện hỗ trợ cho hệ sinh thái tính toán khoa học trong Python. Dù không phải thành phần trung tâm của lớp web, thư viện này hỗ trợ môi trường phân tích dữ liệu và các tác vụ tính toán đi kèm khi hệ thống cần mở rộng sang các thao tác số sâu hơn.

Ứng dụng trong hệ thống:

- Bổ sung cho lớp tính toán số và các thao tác phân tích dữ liệu phụ trợ.
- Tạo nền tảng mở rộng cho các xử lý định lượng hoặc thống kê trong các bước EDA và thực nghiệm.

* **joblib**



joblib là thư viện chuyên dùng cho việc tuần tự hóa và nạp lại các đối tượng Python có kích thước lớn, đặc biệt phù hợp với các artifact học máy. Trong hệ thống này, joblib đóng vai trò quan trọng ở lớp persistence của mô hình.

Ứng dụng trong hệ thống:

- Lưu và nạp lại các model version dưới dạng `model.pkl`.
- Hỗ trợ cơ chế quản lý version mô hình theo từng thư mục artifact riêng biệt.
- Giúp thời gian nạp mô hình cho suy luận runtime nhanh và ổn định hơn.

* **kagglehub**

Kaggle/kagglehub

Python library to access Kaggle resources



kagglehub là thành phần phục vụ bước thu nhận dữ liệu nguồn từ Kaggle. Công cụ này hỗ trợ chuẩn hóa bước thu nhận dữ liệu, nhờ đó giảm phụ thuộc vào thao tác thủ công và nâng cao tính tái lập khi huấn luyện lại mô hình.

Ứng dụng trong hệ thống:

- Tải dataset từ tham chiếu Kaggle đã được cấu hình sẵn.
- Chuẩn hóa bước thu nhận dữ liệu đầu vào để thuận tiện cho việc tái sử dụng và huấn luyện lại.

* **Matplotlib và Seaborn**

matplotlib



Matplotlib là thư viện trực quan hóa nền tảng, còn Seaborn mở rộng thêm lớp trực quan hóa thống kê với phong cách biểu đồ phù hợp cho phân tích dữ liệu. Hai công nghệ này được sử dụng chủ yếu ở pha EDA, phân tích chất lượng dữ liệu và sinh hình phục vụ báo cáo.

Ứng dụng trong hệ thống:

- Tạo các biểu đồ phân phối, quan hệ đặc trưng và trực quan hóa dữ liệu trước huấn luyện.
- Sinh các artifact hình ảnh dùng trong báo cáo, dashboard phân tích và phần giải thích dữ liệu.
- Hỗ trợ data scientist quan sát dữ liệu và đọc hành vi của tập huấn luyện theo hướng trực quan hơn.

* Lưu trữ cục bộ

Lưu trữ cục bộ là tầng storage hiện tại của hệ thống trong phạm vi đề tài. Thay vì sử dụng cơ sở dữ liệu quan hệ hay object storage từ đầu, hệ thống dùng các tệp CSV, JSON và thư mục artifact version để giữ cấu trúc vận hành đơn giản, dễ quan sát và dễ phục hồi.

Ứng dụng trong hệ thống:

- Lưu dữ liệu huấn luyện cục bộ dưới dạng CSV.
- Lưu trạng thái vận hành, tài khoản và active model dưới dạng JSON.
- Lưu các artifact mô hình theo từng thư mục version để thuận tiện cho việc rollback, so sánh và kích hoạt mô hình.

4 Tập dữ liệu, phân tích đặc trưng và pipeline tiền xử lý

4.1 Nguồn dữ liệu và cấu trúc ngữ nghĩa ban đầu

Bộ dữ liệu nền của đề tài được lấy từ Kaggle, cụ thể là dataset **Used Phones & Tablets Pricing Dataset** của tác giả **ahsan81**. Theo mô tả nguồn, dataset này được xây dựng cho các bài toán phân tích giá và dự đoán giá của thiết bị cầm tay đã qua sử dụng; tập dữ liệu gốc có 3.454 bản ghi và 15 trường dữ liệu. Ở góc nhìn nghiệp vụ, mỗi bản ghi biểu diễn một thiết bị đã qua sử dụng cùng với thông tin cấu hình, vòng đời sử dụng và mặt bằng giá tương ứng.

Để mô tả ngắn gọn bộ dữ liệu gốc, có thể tóm tắt như sau:

Thuộc tính	Mô tả
Nguồn dữ liệu	Kaggle – Used Phones & Tablets Pricing Dataset
Đơn vị quan sát	Một thiết bị cầm tay đã qua sử dụng
Quy mô dữ liệu gốc	3.454 bản ghi, 15 trường
Mục tiêu sử dụng	Phân tích giá và xây dựng mô hình dự đoán giá
Dạng bài toán	Hồi quy trên dữ liệu bảng

Ở mức trường dữ liệu, bộ gốc bao gồm các cột: `device_brand`, `os`, `screen_size`, `4g`, `5g`, `rear_camera_mp`, `front_camera_mp`, `internal_memory`, `ram`, `battery`, `weight`, `release_year`, `days_used`, `normalized_used_price` và `normalized_new_price`. Cấu trúc này cho thấy dataset không chỉ chứa thông số kỹ thuật của thiết bị, mà còn chứa trực tiếp hai biến giá ở dạng log-price để phục vụ các bài toán hồi quy.

Để phục vụ hệ thống hiện tại, dữ liệu gốc được đưa về một schema vận hành thống nhất. Tập dữ liệu cục bộ đang được sử dụng tại `Mobile-Phone-Price-Prediction/app/data/sample_phone_data.csv` có 3.454 bản ghi và 13 cột, tức đã lược bỏ hai cột kết nối 4g và 5g. Việc thu gọn này phản ánh đúng định hướng của pipeline hiện hành: chỉ giữ các biến đang thực sự đi vào huấn luyện và suy luận, từ đó làm schema ổn định hơn cho cả nhập liệu, train và runtime. Từ góc nhìn ngữ nghĩa, các trường của dữ liệu có thể được tổ chức thành ba nhóm lớn:

- **Nhóm nhận diện và cấu hình:** `device_brand`, `os`, `screen_size`, `rear_camera_mp`, `front_camera_mp`, `internal_memory`, `ram`, `battery`, `weight`.
- **Nhóm vòng đời sử dụng:** `release_year`, `days_used`.
- **Nhóm giá trị thị trường:** `normalized_used_price`, `normalized_new_price`.

4.2 Chuẩn hóa dữ liệu, xây dựng đặc trưng

Khi nhập bản ghi thủ công hoặc import CSV, dữ liệu đều được đưa qua bước chuẩn hóa schema để bảo đảm đủ cột, đúng tên trường và đúng cách biểu diễn giá trị mục tiêu. Các bước bao gồm:

1. kiểm tra dữ liệu đầu vào có khớp `RAW_SCHEMA` hay không;
2. chuẩn hóa kiểu dữ liệu của các cột số như `screen_size`, `ram`, `battery`, `release_year` và `days_used`;
3. chuyển giá thô về dạng `normalized_used_price` và `normalized_new_price` nếu dữ liệu nhập ở dạng giá gốc;

4. ánh xạ dữ liệu nguồn sang tập đặc trưng huấn luyện thống nhất;
5. tách giữa tập đặc trưng đầy đủ và tập đặc trưng mặc định để phục vụ train theo cấu hình chuẩn hoặc train thử nghiệm.

Sau bước chuẩn hóa, phần feature engineering được thực hiện để tạo ma trận huấn luyện. Trong bước này, dữ liệu gốc được ánh xạ về tập đặc trưng phù hợp với mô hình Random Forest, đồng thời phân tách giữa tập đặc trưng đầy đủ và tập đặc trưng mặc định. Tập đầy đủ cho phép mở rộng khi cần thử nghiệm, còn tập mặc định giữ vai trò cấu hình ổn định cho các lần huấn luyện thông thường.

4.3 Đặc điểm thống kê của dữ liệu, EDA và chiến lược tăng cường mẫu

Trên tập dữ liệu cục bộ hiện tại, phân bố thương hiệu không đồng đều. Năm nhóm xuất hiện nhiều nhất lần lượt là **Others** (502 bản ghi), **Samsung** (341), **Huawei** (251), **LG** (201) và **Lenovo** (171). Điều đó cho thấy dữ liệu nghiêng mạnh hơn về một số nhóm phổ biến, kéo theo nguy cơ mô hình học tốt hơn ở các nhóm đông mẫu nhưng kém ổn định hơn ở các nhóm hiếm.

Ngoài mất cân bằng theo thương hiệu, dữ liệu hiện hành cũng còn giá trị thiếu ở một số cột cấu hình. Đáng chú ý nhất là **rear_camera_mp** thiếu 179 giá trị; các cột như **battery**, **weight**, **internal_memory** và **ram** cũng có thiếu hụt ở mức nhỏ hơn. Điều này cho thấy dataset dùng trong hệ thống không thể được xem là đầu vào đã hoàn toàn sạch, mà cần đi qua bước chuẩn hóa và kiểm tra schema trước khi huấn luyện.

Trạng thái của tập dữ liệu đang dùng có thể tóm tắt ngắn gọn như sau:

Khía cạnh	Quan sát
Phân bố thương hiệu	không đồng đều, nghiêng về một số nhóm phổ biến
Giá trị thiếu	xuất hiện ở một số cột cấu hình phần cứng
Biến giá mục tiêu	đã có sẵn ở dạng log-price
Hàm ý cho pipeline	cần chuẩn hóa dữ liệu và kiểm soát đại diện mẫu

Nhằm phục vụ quá trình phân tích dữ liệu, các biểu đồ trực quan hóa dữ liệu được tổ chức thành ba lớp phân tích chính, trong đó mỗi lớp gắn với một mục tiêu đọc dữ liệu cụ thể:

- **Phân phối đơn biến số:** trong nhóm này, **box plot** hỗ trợ phát hiện ngoại lệ và các khoảng giá trị bất thường, còn **distribution plot** kết hợp KDE giúp nhận diện độ lệch phân phối, vùng mật độ tập trung và khả năng tồn tại nhiều cụm giá trị.



Figure 4.1: Biểu đồ phân phối đơn biến số

- **Quan hệ giữa các biến:** correlation matrix trên toàn bộ các cột số và cho phép dựng scatter plot giữa hai biến bất kỳ, mặc định theo hướng một đặc trưng so với `normalized_used_price`. Ở đây, **correlation matrix** được dùng để nhận diện các cặp biến có quan hệ mạnh nhằm hỗ trợ chọn đặc trưng, trong khi **scatter plot** giúp kiểm tra trực quan mức độ liên hệ giữa giá máy cũ với các biến.

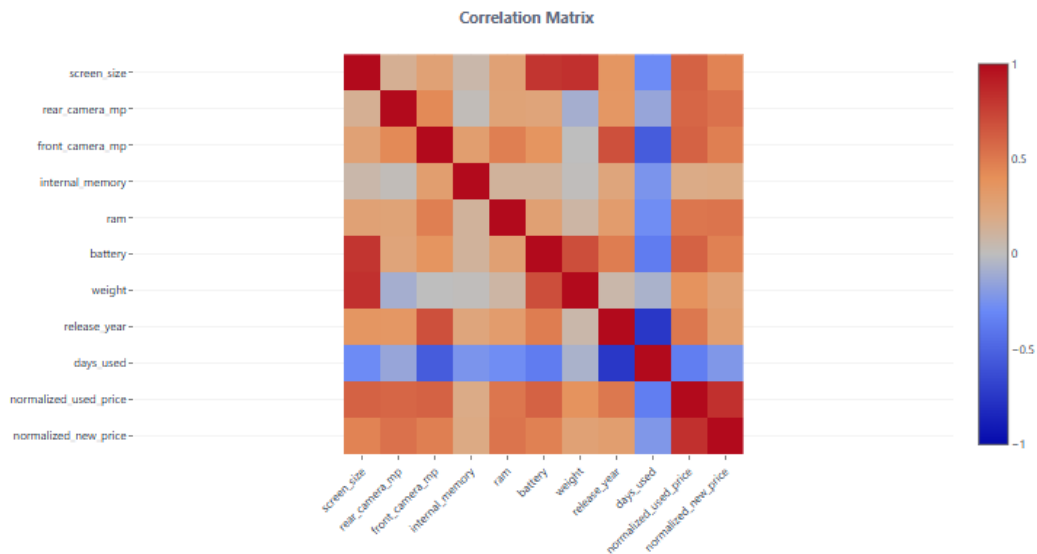


Figure 4.2: Biểu đồ thể hiện quan hệ giữa các đặc trưng.

- **Phân bố theo nhóm:** các đồ thị tần suất của các biến dùng để đọc cấu trúc mẫu của dữ liệu. Nhóm biểu đồ này đặc biệt hữu ích trong việc đánh giá mức độ mất cân bằng dữ liệu như giữa các thương hiệu, hệ điều hành và các mức cấu hình phổ biến.

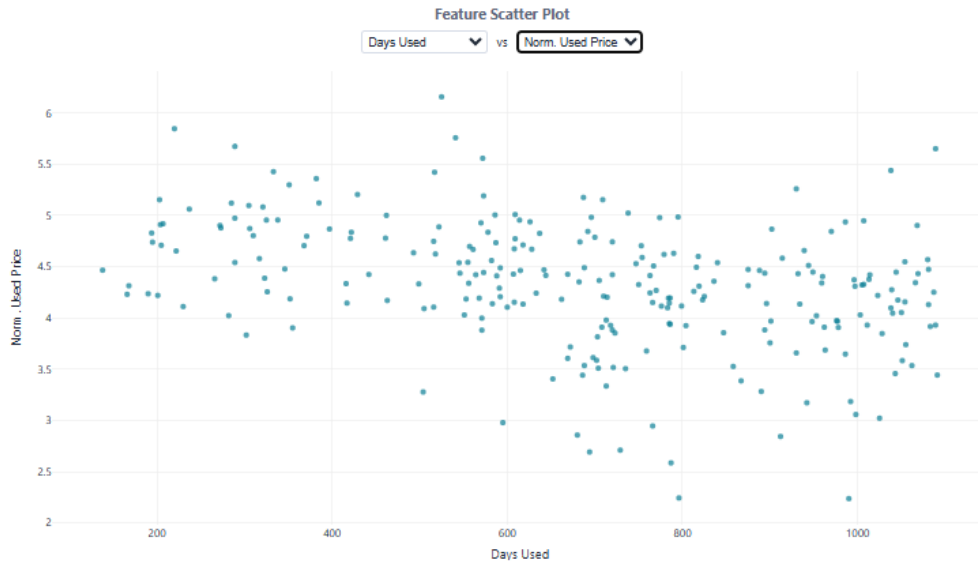


Figure 4.3: Biểu đồ phân bố mẫu và cấu trúc dữ liệu.

Như vậy, các biểu đồ trực quan hóa này sẽ là công cụ hỗ trợ data scientist đưa ra quyết định về làm sạch dữ liệu và xử lý ngoại lệ, lựa chọn hoặc ưu tiên đặc trưng cho mô hình, cũng như xác định có cần tăng cường dữ liệu để bù mất cân bằng mẫu hay không.

Từ đặc điểm đó, phần augmentation được thêm vào như một bước chủ động điều chỉnh phân bố dữ liệu. Cơ chế tăng cường mẫu trong dự án tập trung vào việc sinh dữ liệu iPhone tổng hợp với nhiều hồ sơ phân phối khác nhau. Thay vì sinh ngẫu nhiên theo một cách duy nhất, dữ liệu tổng hợp được tổ chức theo nhiều profile để có thể điều chỉnh mức độ tập trung vào máy đời mới, mức phân bố cân bằng hơn hoặc mức thiên về một nhóm cấu hình cụ thể. Việc giữ nhiều profile như vậy giúp augmentation không trở thành một thao tác cứng, mà là một phần của chiến lược dữ liệu có thể thay đổi theo mục tiêu huấn luyện.

Về mặt thực tiễn, augmentation không nhằm thay thế dữ liệu thật mà nhằm lấp bớt khoảng trống đại diện của một số nhóm thiết bị. Toàn bộ phần này được đặt trực tiếp vào ứng dụng để data scientist có thể điều chỉnh dữ liệu ngay trong luồng vận hành.

Sau khi dữ liệu đã được dựng thành ma trận huấn luyện, pipeline thực hiện chia train-test theo tỷ lệ 80/20. Cách chia này được dùng để tạo ra một mặt bằng đánh giá thống nhất giữa các version mô hình. Dù đây là cấu hình cơ bản, nó vẫn đủ để đối chiếu nhanh các lần huấn luyện trong phạm vi ứng dụng cục bộ.

Xét trên toàn bộ quá trình chuẩn bị dữ liệu, pipeline hiện tại được xây dựng quanh ba yêu cầu: dữ liệu phải có schema thống nhất, phải có khả năng tăng cường khi thiếu đại diện và phải giữ được tính nhất quán khi đi từ nhập liệu đến huấn luyện rồi sang suy luận. Ba yêu cầu này tạo thành phần xương sống của toàn bộ vòng đời dữ liệu trong ứng dụng.

5 Vòng đời mô hình AI và cơ chế huấn luyện

5.1 Tổng quan vòng đời mô hình

Trong quá trình xây dựng dự án, mô hình không được tổ chức như một artifact tĩnh chỉ huấn luyện một lần rồi sử dụng cố định mà toàn bộ luồng xử lý đã được thiết kế theo tư duy vòng đời mô hình, trong đó dữ liệu được cập nhật, mô hình được huấn luyện lại, kết quả được đánh giá, phiên bản mới được lưu trữ, phiên bản đang hoạt động được lựa chọn, và cuối cùng mô hình mới được đưa vào phục vụ suy luận. Cách tổ chức này giúp phần AI gắn trực tiếp với vận hành của ứng dụng thay vì tồn tại như một thành phần tách rời.

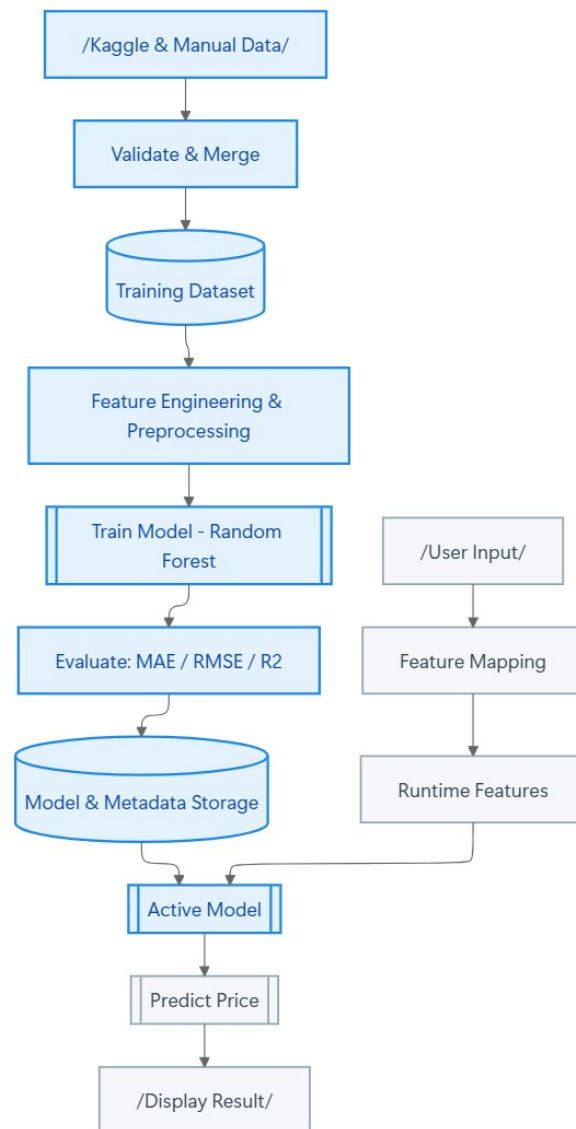


Figure 5.1: Sơ đồ luồng AI pipeline của hệ thống

Với pipeline hiện tại, dữ liệu đầu vào được lấy từ tập Kaggle và có thể được mở rộng bằng các bản ghi thêm thủ công hoặc sinh tự động bởi hệ thống nhằm tăng độ phong phú của tập huấn luyện. Sau bước kiểm tra schema, dữ liệu được hợp nhất và lưu lại thành tập làm việc chung. Từ tập này, pipeline tiếp tục thực hiện tách train/test, tiền xử lý đặc trưng, huấn luyện mô hình Random Forest, tính các độ đo đánh giá, lưu artifact, lưu metadata và cập nhật phiên bản đang hoạt động.

Ở nhánh suy luận, dữ liệu do người dùng nhập từ giao diện không được đưa trực tiếp vào mô hình. Thay vào đó, đầu vào được ánh xạ qua catalog thiết bị và được dựng lại đúng cấu trúc đặc trưng mà mô hình đã dùng trong quá trình huấn luyện. Cách tổ chức này nhằm nâng cao trải nghiệm người dùng: nếu buộc người dùng nhập đầy đủ toàn bộ các đặc trưng như trong tập huấn luyện thì biểu mẫu sẽ rất dài và khó sử dụng. Với hướng tiếp cận hiện tại, người dùng chỉ cần cung cấp một số thông tin bề mặt như model, tình trạng pin hoặc thời gian sử dụng, còn hệ thống sẽ suy ra và điền tiếp các thông số phần cứng liên quan từ catalog thiết bị đã xây dựng sẵn. Nhờ đó, giao diện vẫn gọn, dễ thao tác và gần hơn với cách người dùng thực tế mô tả một chiếc điện thoại cần định giá.

5.2 Cơ chế huấn luyện

Ứng dụng cho phép người vận hành có thể nhập các siêu tham số chính của mô hình, đồng thời bật chế độ chọn feature tùy biến cho từng lần huấn luyện. Cụ thể, hệ thống cho phép giữ bộ feature mặc định hoặc đánh dấu lại từng biến đầu vào như brand, RAM, storage, battery, screen size, camera, release year, days used và một số biến mở rộng khác. Nhờ đó, dashboard không chỉ phục vụ thao tác train model đơn thuần mà còn hỗ trợ so sánh nhiều cấu hình đầu vào khác nhau ngay trong cùng một môi trường vận hành.

Phân tích kỹ thuật tối ưu hóa trong quá trình huấn luyện

Mô hình lõi được chọn cho bài toán là Random Forest Regressor. Với lựa chọn này, phần “tối ưu hóa” trong quá trình huấn luyện không được hiểu theo nghĩa cập nhật trọng số bằng gradient descent như trong mạng nơ-ron, mà được hiểu theo nghĩa lựa chọn cấu trúc cây và điều chỉnh siêu tham số để đạt khả năng tổng quát hóa tốt trên dữ liệu chưa thấy.

Nhóm tham số điều chỉnh chính được đưa ra cho quá trình huấn luyện gồm các siêu tham số của mô hình và cả lựa chọn tập feature đầu vào:

- `n_estimators`: số cây trong rừng.
- `max_depth`: độ sâu cực đại của mỗi cây.
- `min_samples_split`: số mẫu tối thiểu để một nút được phép tách.
- `min_samples_leaf`: số mẫu tối thiểu ở một nút lá.

- **features:** tập đặc trưng được chọn cho lần huấn luyện, có thể dùng bộ mặc định hoặc tùy chỉnh trực tiếp trên dashboard.

Trong quá trình thiết kế, các tham số này được xem như công cụ kiểm soát trực tiếp cả độ phức tạp của mô hình lẫn phạm vi thông tin đầu vào mà mô hình được phép sử dụng. Khi tăng số cây, dự đoán thường ổn định hơn nhưng chi phí huấn luyện và suy luận cũng tăng theo. Khi cho phép cây quá sâu hoặc lá quá nhỏ, mô hình có xu hướng bám sát dữ liệu huấn luyện hơn mức cần thiết. Ở chiều ngược lại, việc thêm hoặc bớt feature cũng làm thay đổi khả năng biểu đạt của mô hình và mức độ rủi ro leakage. Vì vậy, quá trình huấn luyện trong hệ thống được thiết kế theo hướng cho phép thử nghiệm linh hoạt nhiều cấu hình, nhưng vẫn giữ một lối xử lý thống nhất để bảo đảm tính nhất quán giữa các lần train.

5.3 Kiểm soát chất lượng mô hình và tính hợp lệ đặc trưng

Kiểm soát overfitting

Trong hệ thống hiện tại, kiểm soát overfitting được thực hiện bằng các cơ chế phù hợp với mô hình cây thay vì vay mượn các kỹ thuật của deep learning. Trọng tâm nằm ở việc giảm phương sai của ensemble, giới hạn độ phức tạp của từng cây và tránh để mô hình hấp thụ các tín hiệu quá gần với biến mục tiêu.

1. **Ensemble averaging:** lấy trung bình trên nhiều cây để làm mượt dự đoán và giảm phương sai tổng thể.
2. **Giới hạn độ phức tạp cây:** dùng `max_depth`, `min_samples_split` và `min_samples_leaf` để ngăn cây phát triển quá mức.
3. **Kiểm soát phạm vi feature:** cho phép chọn tập đặc trưng huấn luyện nhưng vẫn giữ một bộ mặc định gọn, tránh mở rộng đầu vào không kiểm soát.

Tính hợp lệ của dữ liệu và đặc trưng

Ngoài overfitting theo nghĩa mô hình, hệ thống còn kiểm soát chất lượng dự đoán ở lớp dữ liệu và đặc trưng. Các bước như kiểm tra schema, chuẩn hóa bản ghi và loại bỏ biến có nguy cơ leakage được xem là điều kiện bắt buộc để bảo đảm dữ liệu train và dữ liệu suy luận đều phản ánh đúng tín hiệu nghiệp vụ hợp lệ.

Feature validity và target leakage

Biến `normalized_new_price` không được đưa vào tập mặc định vì có tương quan quá mạnh với giá cần dự đoán. Về mặt kiến trúc, đây là quyết định thuộc *data/feature validity*: mục tiêu là tránh một mô hình cho kết quả đẹp trên tập đánh giá nhưng phụ thuộc vào thông tin không bền vững hoặc khó thu thập ở thời điểm suy luận thực tế.

5.4 Quản trị vòng đời mô hình và vận hành suy luận

Theo phân lớp chuẩn của ML system, toàn bộ các nội dung như metadata, versioning, chọn active model, inference consistency hay fallback runtime đều thuộc *model lifecycle* và *inference serving*.

Metadata huấn luyện

Sau mỗi lần train, hệ thống lưu kèm `metadata.json` bên cạnh `model.pkl`. Metadata ghi nhận version, tập feature, target, chỉ số MAE/RMSE/ R^2 , siêu tham số và thời điểm huấn luyện; nhờ đó mỗi artifact có đủ ngữ cảnh để so sánh, truy vết và giải thích quyết định chọn mô hình.

Versioning mô hình

Các phiên bản được tổ chức theo thư mục `app/model_versions/v1`, `v2`, ..., và mỗi lần train mới tạo ra một version độc lập thay vì ghi đè artifact cũ. Cấu trúc này thuộc lớp *model lifecycle management*, vì chức năng chính của nó là bảo toàn lịch sử, hỗ trợ rollback và cho phép so sánh giữa các lần huấn luyện.

Chọn active model

Một model chỉ đi vào phục vụ suy luận khi được đánh dấu là `active_model_version` trong `app_settings.json`. Cơ chế này tách quyết định vận hành khỏi mã nguồn ứng dụng, cho phép đổi model đang phục vụ như một thao tác quản trị thay vì một thay đổi logic trong pipeline.

Tính nhất quán của suy luận

Ở thời điểm runtime, hàm `predict()` đọc lại cấu trúc đặc trưng đã được lưu cùng preprocessor hoặc regressor để dựng đúng thứ tự cột đầu vào. Đây là cơ chế *inference consistency*: mục tiêu là bảo đảm model luôn nhận đúng schema mà version tương ứng đã dùng trong quá trình huấn luyện.

Cơ chế fallback

Khi không nạp được bất kỳ artifact hợp lệ nào, hệ thống chuyển sang `_mock_predict()` để duy trì khả năng phản hồi tối thiểu. Về mặt kiến trúc, đây là một biện pháp *runtime safety*, giúp ứng dụng không dừng hoàn toàn trong các tình huống lỗi artifact, demo hoặc kiểm thử giao diện.

Mức sẵn sàng vận hành

Ở phạm vi hiện tại, hệ thống đã bao quát được một vòng đời vận hành khép kín: huấn luyện, lưu version, chọn active model và phục vụ suy luận trong cùng một ứng dụng. Tuy nhiên, nếu nâng lên production đầy đủ, cần bổ sung thêm các lớp như audit log, monitoring, kiểm soát đồng thời, backup artifact và quản trị lỗi ở mức dịch vụ.

6 Đánh giá kết quả

6.1 Khung đánh giá chất lượng mô hình

Chất lượng mô hình được theo dõi bằng ba chỉ số chuẩn cho hồi quy: MAE, RMSE và R^2 . Ba chỉ số này được tính trên tập test sau phép chia train-test 80/20 và được ghi lại trong metadata của mỗi version mô hình. Phần định nghĩa toán học, ý nghĩa của từng ký hiệu và lập luận lựa chọn bộ ba chỉ số này đã được trình bày ở Mục 2.4. Trong chương này, ba chỉ số được sử dụng như khung đánh giá thực nghiệm thống nhất để so sánh các version mô hình trong kho hiện hành.

6.2 So sánh tổng quan các version mô hình

Ở giai đoạn đánh giá, từng model không được xem như một artifact độc lập mà được đặt trong cùng một không gian thực nghiệm. Vì vậy, trước khi đi vào phân tích chi tiết một version cụ thể, cần nhìn toàn bộ các model hiện hành như một chuỗi thí nghiệm, trong đó mỗi version phản ánh một lựa chọn khác nhau về dữ liệu, tập feature hoặc cấu hình huấn luyện.

Tiêu chí so sánh

Việc so sánh các version được thực hiện trên cùng ba chỉ số MAE, RMSE và R^2 . Trong giao diện quản trị hiện tại, tiêu chí xếp hạng chính để gợi ý “best choice” là R^2 cao nhất; tuy nhiên, trong phân tích, cả ba chỉ số vẫn cần được đọc đồng thời để tránh kết luận thiên lệch theo một thước đo duy nhất.

Bảng tổng hợp các version hiện hành

Version	Records	Features	R^2	MAE	RMSE
v1	3567	6	0.9217	23.1648	39.6847
v2	3568	6	0.8954	21.5228	37.3287
v3	4454	6	0.9556	34.7660	60.2850
v4	4454	8	0.9610	31.8551	54.9547
v5	3454	8	0.6983	19.1244	28.4388

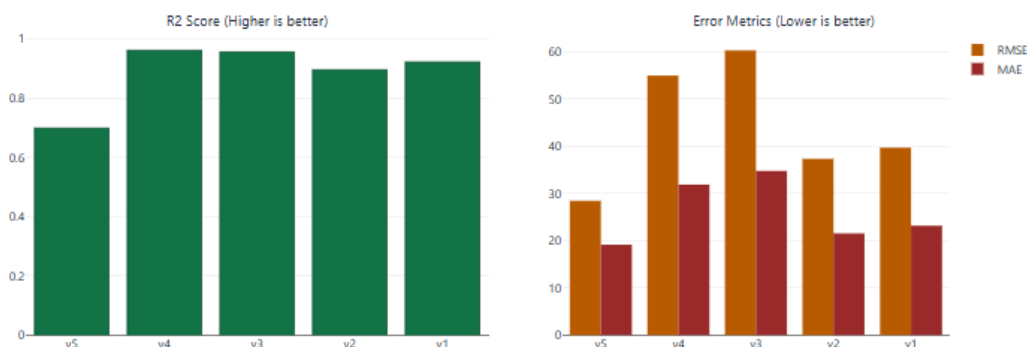


Figure 6.1: Đồ thị tổng hợp các chỉ số đánh giá chất lượng của các version mô hình.

Bảng trên cho thấy các version hiện tại không tạo thành một trật tự đơn điệu trên mọi chỉ số. Nếu xét theo R^2 , v4 là phiên bản tốt nhất; nếu xét theo MAE và RMSE, v5 lại có sai số tuyệt đối thấp hơn. Điều này cho thấy việc đánh giá mô hình trong hệ thống phải được hiểu như một bài toán đa tiêu chí, thay vì một phép xếp hạng tuyến tính đơn giản.

Theo tiêu chí xếp hạng đang được dashboard sử dụng, phiên bản được đánh giá tốt nhất trong kho model hiện hành là v4, tức version có R^2 cao nhất. v4 được chọn làm đối tượng phân tích trọng tâm vì đây là model thể hiện tốt nhất khả năng nắm bắt cấu trúc biến thiên của dữ liệu mục tiêu trên tập đánh giá.

Về mặt định lượng, đây là version có mức giải thích phương sai cao nhất trong toàn bộ kho model hiện tại.

- **Về khả năng giải thích dữ liệu:** $R^2 \approx 0.961$ cho thấy mô hình đã học được phần lớn cấu trúc biến thiên của giá trên tập test. Đây là cơ sở chính để v4 được dashboard xếp ở vị trí tốt nhất.
- **Về nguyên nhân cải thiện:** so với các version trước, v4 hưởng lợi từ hai thay đổi quan trọng là tập dữ liệu lớn hơn và feature set được mở rộng từ 6 lên 8 biến. Hai biến bổ sung là `release_year` và `days_used` đưa thêm thông tin về vòng đời và mức khấu hao của thiết bị.
- **Về giới hạn của kết quả:** v4 không phải model có MAE hay RMSE thấp nhất. Điều này cho thấy khả năng giải thích phương sai tổng thể và độ lớn sai số tuyệt đối không hoàn toàn đồng biến trong các thí nghiệm hiện có, v4 được xem là model tốt nhất theo tiêu chí xếp hạng hiện hành của hệ thống, chứ chưa thể xem là phiên bản tối ưu tuyệt đối trên mọi góc nhìn đánh giá.

Tầm quan trọng đặc trưng của version v4

Metadata của v4 cho thấy thứ tự feature importance giảm dần là: **brand**, **storage**, **screen_size**, **release_year**, **battery_capacity**, **ram**, **days_used**, **camera_mp**.

- **Nhóm tác động mạnh nhất:** **brand** đứng đầu, cho thấy dữ liệu thị trường hiện tại vẫn mang dấu ấn thương hiệu rất rõ trong quá trình hình thành giá.
- **Nhóm phân khúc kỹ thuật:** **storage**, **screen_size** và **ram** giúp mô hình phân biệt các cụm thiết bị theo cấu hình và phân khúc sử dụng.
- **Nhóm vòng đời sản phẩm:** **release_year** và **days_used** bổ sung thông tin trực tiếp về thế hệ sản phẩm và mức khấu hao theo thời gian.
- **Nhóm giá trị sử dụng thực tế:** **battery_capacity** và **camera_mp** không phải các biến chi phối mạnh nhất, nhưng vẫn góp phần phản ánh chất lượng sử dụng và sức hấp dẫn của thiết bị trên thị trường.
- **Nhận xét tổng quát:** mô hình không chỉ học từ thông số phần cứng, mà còn khai thác được đồng thời tín hiệu thương hiệu, vòng đời công nghệ và mức độ hao mòn của sản phẩm.

6.3 Thảo luận kết quả

Sau khi đặt các version vào cùng một khung so sánh và chọn ra model được đánh giá tốt nhất theo tiêu chí hiện hành, bước tiếp theo là diễn giải kết quả trong bối cảnh vận hành của hệ thống. Mục tiêu ở đây không chỉ là trả lời “model nào tốt hơn”, mà còn là làm rõ hệ quả của các kết quả đó đối với việc triển khai và sử dụng mô hình trong ứng dụng.

Ý nghĩa của việc so sánh version

Cơ chế so sánh version cho phép trả lời các câu hỏi thực nghiệm cốt lõi: mở rộng feature set có giúp cải thiện mô hình hay không, tăng dữ liệu tổng hợp có thực sự nâng chất lượng, và thay đổi cấu hình huấn luyện tác động ra sao đến từng chỉ số. Nhờ đó, quá trình phát triển model được tổ chức như một chuỗi thí nghiệm có kiểm soát, thay vì chỉ là tập hợp các lần train rời rạc.

Ý nghĩa đối với quyết định triển khai

Kết quả hiện tại cho thấy việc chọn model để phục vụ runtime không nhất thiết trùng với model có R^2 cao nhất. Đây là một điểm quan trọng trong thiết kế hệ thống: môi trường vận hành có thể ưu tiên một version ổn định hơn theo sai số tuyệt đối hoặc phù hợp hơn với phân phối dữ liệu đang khai thác, trong khi môi trường đánh giá học thuật có thể ưu tiên version có năng lực giải thích mạnh hơn về mặt thống kê.

Phạm vi sử dụng của kết quả hiện tại

Trong trạng thái hiện tại, pipeline đã bao quát các lớp chính gồm dữ liệu, huấn luyện, versioning, dashboard quản trị và suy luận trực tiếp. Kết quả của mô hình được dùng như công cụ hỗ trợ tham chiếu giá, chưa được đặt như một cơ chế quyết định hoàn toàn tự động. Cách đặt phạm vi sử dụng như vậy phù hợp với bản chất của bài toán định giá khi dữ liệu vẫn còn giới hạn và nhiều biến ngữ cảnh chưa được đưa vào mô hình.

Phụ lục kỹ thuật

A Cấu trúc hiện thực của hệ thống và quan hệ giữa các module

A.1 Điểm vào và lớp điều phối ứng dụng

Điểm vào của ứng dụng

Tệp `Mobile-Phone-Price-Prediction/app.py` ở thư mục gốc chỉ đảm nhiệm một vai trò ngắn gọn: import hàm `create_app()` từ `app.app`, sau đó tạo đối tượng `app` và chạy Flask trên địa chỉ `127.0.0.1:8000` ở chế độ debug. Thiết kế này tách rời rõ *entrypoint* và *application factory*, nhờ đó việc thay đổi cách khởi chạy sau này thuận lợi hơn.

Vai trò của `app/app.py`

Ở lớp ứng dụng web, `app/app.py` là trung tâm điều phối chính. Tệp này tạo Flask app, cấu hình khóa bí mật, sinh danh sách thương hiệu và model từ `DEVICE_MODEL_BRANCH_MAP`, định nghĩa hàm dựng đặc trưng runtime và khai báo các route trọng yếu như `home`, `login`, `logout`, `admin`, `data-science` và route phục vụ `report`. Về bản chất, đây là nơi nối giao diện HTML, lớp controller nghiệp vụ và lớp mô hình dự đoán.

Logic dựng feature runtime

Hàm `_build_features_from_form()` là một điểm then chốt trong thiết kế hệ thống vì nó nối thế giới người dùng với thế giới mô hình. Hàm này thực hiện bốn việc chính:

1. kiểm tra hợp lệ của `branch` và `model`;
2. tra cứu template của model trong `DEVICE_MODEL_BRANCH_MAP`;
3. tính các biến suy diễn như `battery_capacity` từ mức sức khỏe pin;
4. trả về đồng thời *feature dictionary* cho mô hình và `ui_info` cho giao diện.

Nhờ lớp logic này, người dùng không cần nhập toàn bộ thông số phần cứng như trong dữ liệu huấn luyện. Đầu vào suy luận vì vậy gọn hơn, nhất quán hơn và ít sai ngữ nghĩa hơn.

A.2 Các module nghiệp vụ và hạ tầng

Nhóm xác thực và phân quyền

`auth_controller.py` được giữ ở cấu trúc tối giản để phục vụ đúng bài toán phân quyền. Nó đọc thông tin người dùng từ `app_settings.json`; nếu tệp chưa có dữ liệu quản trị thì sinh bộ người dùng mặc định từ `credentials.py`. Hàm `validate_login()`

đối chiếu tài khoản, mật khẩu và trả về role hợp lệ. Cách làm này đủ gọn cho phạm vi đồ án, dù chưa đạt chuẩn production do mật khẩu còn được lưu plaintext trong JSON.

Nhóm điều phối dữ liệu, huấn luyện và quản trị

`admin_controller.py` là mô-đun lớn nhất ở lớp điều phối nghiệp vụ. Các trách nhiệm chính có thể gom thành bảy nhóm: huấn luyện mô hình, sinh dữ liệu tổng hợp, tóm tắt dữ liệu, chuẩn hóa và nhập dữ liệu, quản trị người dùng, quản trị phiên bản mô hình và tạo payload trực quan hóa. Việc một mô-đun cùng lúc bao phủ người dùng, dữ liệu và mô hình cho thấy hệ thống đã đạt mức hoàn chỉnh về chức năng, nhưng cũng chỉ ra đây là khu vực cần ưu tiên tách nhỏ nếu hệ thống tiếp tục mở rộng.

Nhóm người dùng và metadata cấu hình

`user_controller.py` chịu trách nhiệm cho các thao tác liên quan tới tài khoản người dùng, trong khi `settings.py` đóng vai trò lớp cấu hình dùng chung cho đường dẫn dữ liệu, thư mục model và các tham số vận hành. Hai mô-đun này không trực tiếp thực hiện suy luận, nhưng tạo nền ổn định để các chức năng quản trị và huấn luyện vận hành nhất quán.

Nhóm mô hình và dữ liệu nền

`phone_price_model.py` là lớp lõi của phần machine learning. Nó phụ trách nạp model active, tiền xử lý đầu vào, dự đoán và fallback khi model thật không sẵn sàng. Bên cạnh đó, `device_catalog.py` đóng vai trò nguồn tri thức tĩnh để dựng feature runtime từ model điện thoại, còn `data_utils.py` cung cấp các tiện ích xử lý dữ liệu lặp lại trong pipeline. Ba thành phần này kết hợp lại tạo nên trục kết nối giữa dữ liệu, catalog và mô hình dự đoán.

B Rủi ro, giới hạn và kiểm soát chất lượng của hệ thống

B.1 Rủi ro dữ liệu và rủi ro đánh giá mô hình

Rủi ro dữ liệu gốc và dữ liệu tổng hợp

Rủi ro nền tảng nhất của hệ thống nằm ở chất lượng dữ liệu. Dữ liệu thiết bị cũ ngoài thực tế thường không đồng nhất về cách ghi nhận cấu hình, tình trạng pin, giá tham chiếu và thời điểm giao dịch. Khi dữ liệu gốc đã không ổn định, mô hình dễ học các quan hệ chỉ đúng cục bộ. Rủi ro này tăng thêm khi hệ thống sử dụng dữ liệu tổng hợp để bù thiếu cho một số phân khúc, đặc biệt là nhóm Apple. Nếu profile sinh dữ liệu không phản ánh đúng phân phối thị trường, mô hình có thể được cải thiện trên chỉ số tổng thể nhưng lại lệch ở mức hành vi thực tế.

Vì vậy, dữ liệu tổng hợp chỉ nên được xem là công cụ bù đắp có kiểm soát, không phải

thay thế lâu dài cho dữ liệu quan sát thật. Một hướng cải tiến phù hợp là gắn nguồn gốc bản ghi ngay từ lúc nhập dữ liệu để theo dõi tác động của từng nhóm lên chất lượng mô hình.

Rủi ro leakage và sai lệch đánh giá

Mã nguồn hiện đã có ý thức loại `normalized_new_price` khỏi tập đặc trưng mặc định. Tuy nhiên, vì dashboard cho phép cấu hình feature tùy chỉnh, về lý thuyết vẫn có khả năng huấn luyện mô hình với biến này. Khi đó, chỉ số đánh giá có thể tốt lên theo cách giả tạo. Đây là dạng *evaluation leakage* phát sinh từ quyền cấu hình hơn là từ lỗi tiền xử lý thuần túy.

Một biện pháp phù hợp hơn cho môi trường nghiên cứu là xác định rõ danh sách feature cấm, hoặc ít nhất gắn cảnh báo rõ trong giao diện khi người dùng chọn các biến có khả năng gây leakage.

B.2 Rủi ro vận hành, triển khai và bảo mật

Tính nhất quán của runtime

Ở lớp triển khai, mô hình được nạp động từ version active mỗi khi khởi tạo `PhonePriceModel`. Cách làm này tạo tính linh hoạt cho việc thay đổi model, nhưng cũng mở ra rủi ro: nếu version active bị xóa, ghi đè sai hoặc không nạp được, hệ thống có thể chuyển sang hành vi fallback. Về mặt availability, đây là cách xử lý chấp nhận được; về mặt vận hành, nó lại tạo nguy cơ người dùng không nhận ra đầu ra đang đến từ `_mock_predict()` thay vì mô hình thật.

Do đó, trạng thái nạp model nên được ghi log rõ ràng và hiển thị minh bạch cho admin hoặc data scientist.

Rủi ro bảo mật và quản trị

Từ góc nhìn bảo mật, hệ thống hiện còn một số giới hạn rõ ràng: mật khẩu lưu plaintext trong JSON, secret key mặc định là chuỗi tĩnh nếu không có biến môi trường, chưa có rate limit cho login và chưa có audit log cho các hành động nhạy cảm như đổi active model hay thay đổi người dùng. Những điểm này không làm mất giá trị của đồ án trong bối cảnh chạy localhost, nhưng cần được nêu rõ để tránh đánh giá hệ thống như một sản phẩm production-ready về bảo mật.

Rủi ro diễn giải kết quả bởi người dùng

Một rủi ro thường bị xem nhẹ là rủi ro diễn giải. Mô hình dự đoán giá rất dễ bị hiểu thành “giá đúng tuyệt đối”, trong khi kết quả thực chất chỉ là mức tham chiếu phụ thuộc mạnh vào dữ liệu lịch sử và chưa bao quát các yếu tố như ngoại hình máy, phụ kiện đi kèm hay khu vực giao dịch. Nếu không có lớp giải thích phù hợp, người dùng cuối có thể đặt kỳ vọng sai vào hệ thống.

B.3 Kiểm soát chất lượng hiện có và hướng tăng cường

Các cơ chế kiểm soát hiện có

Dù còn nhiều giới hạn, hệ thống hiện đã có một lớp kiểm soát chất lượng tối thiểu gồm: kiểm tra schema dữ liệu trước khi huấn luyện, kiểm tra giá trị dương của `used_price` và `new_price`, kiểm tra version tồn tại trước khi kích hoạt hoặc xóa, kiểm tra role trước khi truy cập chức năng nhạy cảm và chặn việc xóa admin cuối cùng. Các cơ chế này cho thấy hệ thống không chỉ tập trung vào happy path mà đã bắt đầu xử lý các trường hợp lỗi phổ biến.

Các chỉ dấu chất lượng nên bổ sung

Nếu hệ thống được mở rộng tiếp, các chỉ dấu chất lượng nên được tăng cường theo ba hướng: độ tin cậy thống kê, khả năng giám sát vận hành và mức độ công bằng giữa các nhóm thiết bị. Ở góc độ thống kê, cần bổ sung cross-validation và phân tích sai số theo từng phân khúc. Ở góc độ vận hành, cần có drift detection, logging đầy đủ cho train/import/activate model và test cho các luồng chính. Ở góc độ công bằng, nên theo dõi sai số có điều kiện theo thương hiệu hoặc dải giá, ví dụ:

$$\text{MAE}_{\text{brand}=b} = \frac{1}{n_b} \sum_{i:\text{brand}_i=b} |y_i - \hat{y}_i|.$$

Nếu một số thương hiệu có sai số cao bất thường, dữ liệu hoặc chiến lược augmentation cần được xem xét lại.

C Triển khai, vận hành, bảo trì và định hướng phát triển

C.1 Khởi chạy hệ thống và môi trường thực thi

Mã nguồn hiện tại cung cấp script `start.sh` hỗ trợ tạo môi trường ảo và cài dependency, các dependency cốt lõi bao gồm:

- pandas, numpy cho xử lý dữ liệu;
- scikit-learn cho pipeline và Random Forest;
- kagglehub cho tải dữ liệu;
- Flask cho web app;
- scipy, matplotlib, seaborn cho EDA và trực quan hóa;
- joblib cho tuần tự hóa mô hình.

C.2 Artifact vận hành và quy trình bảo trì mô hình

Trong vận hành, các artifact quan trọng cần được quản lý gồm:

1. `sample_phone_data.csv`: nguồn dữ liệu huấn luyện hiện hành;
2. `model.pkl`: artifact mô hình đã huấn luyện;
3. `metadata.json`: mô tả ngữ cảnh của mỗi model version;
4. `app_settings.json`: trạng thái active model và tài khoản quản lý;
5. `reports/*.png`: artifact trực quan hóa phục vụ dashboard hoặc báo cáo.

Trong vận hành, dữ liệu và metadata cần được sao lưu đồng thời thay vì chỉ giữ model pickle. Nếu mất metadata, bối cảnh huấn luyện sẽ khó được tái dựng đầy đủ.

Từ thiết kế hiện tại, quy trình bảo trì mô hình được tổ chức theo các bước sau:

1. Định kỳ kiểm tra phân phối dữ liệu và các thống kê mô tả;
2. Nhập thêm dữ liệu mới từ CSV hoặc bản ghi thủ công;
3. Nếu cần, sinh thêm dữ liệu tổng hợp theo hồ sơ phù hợp;
4. Huấn luyện phiên bản mới với siêu tham số xác định;
5. So sánh version mới với active version hiện tại;
6. Chỉ chuyển active model khi version mới có kết quả tốt hơn hoặc phù hợp hơn;
7. Giữ lại ít nhất một version cũ ổn định để rollback.

Quy trình này giữ đúng tinh thần versioned deployment trong machine learning trên một nền tảng triển khai gọn nhẹ.

C.3 Các hướng mở rộng, nghiên cứu

Nếu tiếp tục phát triển trong ngắn hạn, các hướng mở rộng trực tiếp nhất gồm mở rộng catalog thiết bị, bổ sung thêm trường dữ liệu như tình trạng ngoại hình, tình trạng pin thực, số SIM, hỗ trợ 5G hoặc khu vực giao dịch, thêm đánh giá chéo và tối ưu lớp suy luận bằng cách cache active model trong bộ nhớ. Các hướng này có thể được thực hiện mà không phải thay đổi triệt để kiến trúc nền.

Ngoài ra, phần kiến trúc có thể được mở rộng theo hướng tách database người dùng và metadata khỏi JSON, tách service huấn luyện khỏi web app, thêm API riêng cho suy luận, xây dựng cơ chế model registry hoặc artifact registry, và bổ sung logging tập trung

đi kèm dashboard giám sát vận hành. Những hướng đi này sẽ làm kiến trúc phức tạp hơn, đồng thời mở đường cho giai đoạn sản phẩm hóa nội bộ.

Về mặt nghiên cứu, phần nền tảng hiện tại cũng cho phép tiếp tục mở rộng theo nhiều hướng: so sánh Random Forest với Gradient Boosting, XGBoost hoặc CatBoost trên cùng pipeline; nghiên cứu ảnh hưởng của các hồ sơ sinh dữ liệu tổng hợp tới từng nhóm thương hiệu; xây dựng mô hình ước lượng bất định để dự đoán khoảng giá thay vì giá trị điểm; bổ sung explainability sâu hơn bằng SHAP hoặc permutation importance; và tổ chức lại cơ chế continual learning hoặc periodic retraining. Các hướng này đều có cơ sở trực tiếp từ hệ thống hiện tại vì versioning, feature control và khả năng train lại đã được đặt sẵn trong pipeline.

D Từ điển dữ liệu, ánh xạ đặc trưng và tham chiếu nghiệp vụ

D.1 Dữ liệu huấn luyện và ánh xạ đặc trưng

Trong một hệ thống học máy triển khai thực tế, dữ liệu không chỉ là “một bảng có nhiều cột” mà còn là hạ tầng ngữ nghĩa của toàn bộ mô hình. Nếu người vận hành không hiểu đúng ý nghĩa của từng trường, đơn vị đo, miền giá trị hợp lệ và mối liên hệ giữa trường đó với biến mục tiêu, pipeline rất dễ suy giảm chất lượng ngay cả khi phần hiện thực phần mềm vẫn vận hành đúng. Vì vậy, một từ điển dữ liệu chi tiết là thành phần quan trọng của báo cáo kỹ thuật.

Phần này xây dựng lớp tham chiếu dữ liệu theo bốn câu hỏi trung tâm:

- dữ liệu là gì;
- mỗi trường đo lường điều gì;
- trường được xử lý ra sao trong pipeline;
- người dùng cần hiểu như thế nào để nhập dữ liệu và diễn giải mô hình một cách chính xác.

Trường	Loại biến	Đơn vị	Diễn giải chi tiết
device_brand	Danh mục	Không áp dụng	Thương hiệu thiết bị. Trong thực tế phản ánh vừa yếu tố kỹ thuật vừa yếu tố cảm nhận thị trường.
os	Danh mục	Không áp dụng	Hệ điều hành thiết bị. Trong dữ liệu hiện có chủ yếu là Android, iOS, Windows và Others.
screen_size	Liên tục	cm	Kích thước đường chéo màn hình theo centimet trong tập gốc; được đổi về inch trong bước huấn luyện.
rear_camera_mp	Liên tục	MP	Độ phân giải camera sau. Chỉ là chỉ báo gần đúng cho khả năng camera, không đo toàn diện chất lượng ảnh.
front_camera_mp	Liên tục	MP	Độ phân giải camera trước.
internal_memory	Liên tục/bán rời rạc	GB	Bộ nhớ trong. Một trong các biến dự báo mạnh nhất trong metadata hiện có.
ram	Liên tục/bán rời rạc	GB	Dung lượng RAM của thiết bị.
battery	Liên tục	mAh	Dung lượng pin danh nghĩa của thiết bị.
weight	Liên tục	gram	Khối lượng thiết bị. Biến này hiện không nằm trong tập đặc trưng mặc định.
release_year	Thứ bậc/số nguyên	năm	Năm phát hành của thiết bị. Gián tiếp phản ánh thế hệ công nghệ.
days_used	Số nguyên	ngày	Số ngày thiết bị đã qua sử dụng. Biến này đóng vai trò thước đo khấu hao.
normalized_used_price	Liên tục	log-price	Logarit tự nhiên của giá máy cũ. Được mũ hóa lại khi huấn luyện.
normalized_new_price	Liên tục	log-price	Logarit tự nhiên của giá máy mới. Có nguy cơ target leakage nếu dùng trực tiếp.

Bảng trên cho thấy mỗi trường không chỉ là một cột dữ liệu, mà còn là một giả định ngữ nghĩa về thị trường thiết bị cũ. Từ đó có thể rút ra hai điểm chính:

- mỗi trường đều mang một nội hàm nghiệp vụ cụ thể;

- nếu giả định ngữ nghĩa của trường sai hoặc thiếu, chất lượng mô hình có thể suy giảm ngay cả khi thuật toán giữ nguyên.

Ánh xạ từ trường nguồn sang đặc trưng mô hình

Hệ thống không huấn luyện trực tiếp trên RAW_SCHEMA. Thay vào đó, nó ánh xạ thành một không gian đặc trưng dễ hiểu hơn đối với mô hình và người vận hành. Bảng sau mô tả quan hệ này.

Trường nguồn	Đặc trưng mô hình	Phép biến đổi
device_brand	brand	Đổi tên, giữ nguyên giá trị chuỗi.
os	os	Giữ nguyên, chỉ dùng khi feature set cho phép.
internal_memory	storage	Đổi tên.
battery	battery_capacity	Đổi tên.
rear_camera_mp	camera_mp	Đổi tên.
screen_size	screen_size	Chuyển đổi từ cm sang inch bằng cách chia cho 2.54.
front_camera_mp	front_camera_mp	Giữ nguyên nếu được chọn.
weight	weight	Giữ nguyên nếu được chọn.
release_year	release_year	Giữ nguyên.
days_used	days_used	Giữ nguyên.
normalized_new_price	normalized_new_price	Giữ nguyên nếu người dùng chọn feature tùy chỉnh.
normalized_used_price	target y	Mũ hóa thành $y = \exp(x)$, với x là <code>normalized_used_price</code> .

Giữa *feature for training* và *feature for storage* có một lớp tách biệt rõ ràng. Cách tổ chức này làm pipeline linh hoạt hơn, đồng thời yêu cầu lớp ánh xạ phải được hiểu đúng trong quá trình vận hành.

D.2 Dữ liệu suy luận và ràng buộc đầu vào

Dữ liệu mà người dùng nhập trong giao diện không trùng hoàn toàn với dữ liệu huấn luyện. Trên giao diện, người dùng chủ yếu thao tác với các biến sau:

- branch/hãng,
- model,
- storage,

- ram (trong một số trường hợp),
- days_used,
- battery_health.

Các biến còn lại như hệ điều hành, kích thước màn hình, camera, năm phát hành và giá mới chuẩn hóa được điền gián tiếp từ catalog. Cách tổ chức này có hai ưu điểm chính:

- giảm gánh nặng nhập liệu cho người dùng;
- giữ đầu vào suy luận gọn hơn và đồng nhất hơn.

Tuy nhiên, cách tiếp cận này cũng đặt ra một yêu cầu quan trọng: catalog thiết bị phải đúng và được cập nhật nhất quán.

Biến suy luận do hệ thống dẫn xuất

Một chi tiết rất hay của hệ thống là biến `battery_health` không tồn tại trong dữ liệu huấn luyện gốc, nhưng lại được dùng tại thời điểm suy luận để điều chỉnh `battery_capacity`. Công thức nội bộ có thể viết dưới dạng:

$$\text{battery_capacity_runtime} = \max \left(1000, \left\lfloor \text{battery_base} \times \frac{\text{battery_health_pct}}{100} \right\rfloor \right).$$

Cơ chế này có ý nghĩa nghiệp vụ rõ ràng ở hai khía cạnh:

- mô hình gốc không cần học trực tiếp từ một trường riêng tên là “battery health” ;
- hệ thống vẫn phản ánh được mức suy giảm thực tế của pin thông qua dung lượng pin hiệu dụng.

Nói cách khác, người dùng có thể cung cấp một yếu tố xuống cấp thực tế của thiết bị mà không làm thay đổi schema dữ liệu huấn luyện cốt lõi.

Ràng buộc đầu vào và miền giá trị hợp lệ

Từ thiết kế kiểm tra đầu vào của hệ thống, có thể rút ra miền giá trị hợp lệ của một số biến nhập như sau:

Biến	Miền giá trị	Nguồn kiểm tra
days_used	từ 1 đến 5×365	<code>_parse_int(..., 1, 5*365)</code>
storage	từ 8 đến 1024	kiểm tra trong hàm dựng feature
ram	từ 1 đến 32 (với máy không phải iPhone)	kiểm tra trong hàm dựng feature
battery_health	từ 1 đến 100, mặc định 80 nếu trống/unknown	logic xử lý form
used_price	> 0	kiểm tra khi nhập dữ liệu
new_price	> 0	kiểm tra khi nhập dữ liệu

Việc xác định rõ miền giá trị của từng biến là cần thiết vì ba lý do:

- hỗ trợ kiểm thử đầu vào;
- hỗ trợ viết tài liệu hướng dẫn người dùng;
- giảm nguy cơ dữ liệu bất thường đi vào pipeline suy luận.

Đặc tính đo lường của các biến

Một sai lầm thường gặp trong mô tả dữ liệu là xem mọi trường số đều là biến liên tục “thuần túy”. Trong phần dữ liệu hiện tại, một số biến tuy lưu dưới dạng số thực nhưng thực chất là bán rời rạc hoặc biến thứ bậc. Ví dụ:

- **ram** thường nhận các mức như 2, 3, 4, 6, 8, 12.
- **internal_memory** nhận các mức 32, 64, 128, 256, 512, 1024.
- **release_year** là biến thứ bậc theo thời gian.
- **days_used** là biến đếm.

Các đặc điểm trên giải thích vì sao mô hình cây phù hợp với bài toán này:

- mô hình xử lý tự nhiên các biến dạng ngưỡng và phân mảnh;
- mô hình không đòi hỏi giả định tuyến tính tron giữa đầu vào và đầu ra như nhiều phương pháp khác.

D.3 Metadata mô hình

Ngoài dữ liệu huấn luyện, pipeline còn sinh một loại “dữ liệu vận hành” quan trọng là metadata mô hình. Từ điển cho metadata được mô tả như sau:

Trường metadata	Kiểu	Ý nghĩa
version	số nguyên	Số hiệu version mô hình.
records	số nguyên	Số bản ghi dùng cho lần huấn luyện này.
kaggle_dataset	chuỗi	Tham chiếu nguồn dữ liệu gốc.
features	danh sách	Tập đặc trưng dùng trong huấn luyện.
target	chuỗi	Mô tả biến mục tiêu.
mae, rmse, r2	số thực	Các chỉ số đánh giá.
trained_at	chuỗi thời gian	Mốc thời gian huấn luyện.
feature_importances	dictionary	Mức quan trọng tương đối của các đặc trưng.